

变物性与径向收缩条件下圆柱药材热湿耦合模型

摘要

针对圆柱药材在时变烘房中的升温、失水和径向收缩过程，本文以干基含水率守恒为主线建立径向热湿输运模型。固定几何采用圆柱坐标，收缩几何通过材料坐标映射到固定参考域；烘房温度、环境水分量与实测半径由分段线性函数输入。空间离散采用环形有限体积法，时间推进采用 BDF 方法。

问题一采用常热物性和含水率相关扩散率。30 min 时，中心与表面温度分别为 33.5753 °C 和 36.7856 °C，中心含水率为 2.5500，表面降至 1.5102 kg/kg。热量已传入主体区域，而水分变化仍集中在表层。

问题二代入附录三变物性，建立由物性系数介导的双向热湿反馈模型。3 h 时，中心与表面含水率分别为 1.7662 和 1.0081；此时温场已接近环境平台，含水率梯度仍然明显，因而不能用温度稳定判断干燥完成。

问题三以全域最大含水率定义终止事件，得到等号临界时刻 57.4740 h；首个严格低于阈值的整数秒为 206907 s。若改用截面平均值或表面值判停，将分别提前约 21.45 h 和 44.92 h。终点对扩散率尺度最敏感，后期干燥主要受内部输运制约。

问题四令 $\xi = r/R(t)$ 随材料运动，将移动域转化为固定域。临界时刻为 51.0920 h，此时半径为 1.2000 cm。四组条件对照表明，问题三与问题四相差的 6.3820 h 同时受到物性和几何的影响；在给定外生半径轨迹下，两者还存在明显交互。

解析基线、网格加密、独立有限元和累计守恒检验均支持上述数值结果。问题三、四对扩散率尺度的无量纲灵敏度分别为 -0.8931 和 -0.8885；相比数值离散，平衡含水率映射、长期环境和经验物性对实际预测的影响更值得关注。

关键词：药材烘干；热湿耦合；有限体积法；移动边界；全域事件

一、引言

1.1 问题背景

热风烘干通过外界对流换热提高药材温度，并通过表面传质持续移走水分。热量与水分在材料内部的传播速率不同，且物性会随温度和含水率变化；在长时间烘干中，材料收缩还会改变传递距离和面积体积比。因此，只给出平均温度或经验干燥曲线，难以回答“内部各处何时均达到安全含水率”这一问题。热质耦合、变物性与收缩几何均是干燥建模中的常见关键因素 [1–3]。

题目给出长度 25 cm、初始半径 2 cm 的圆柱药材，初温为 28 °C，初始干基含水率为 2.55 kg/kg；附件给出烘房温度、环境水分量以及随时间变化的药材半径。本文的任务不是拟合一条总体曲线，而是求解空间分布并据此判定全域达标时刻。

1.2 问题重述

问题 1：采用附录二物性，建立预热阶段的温度场和含水率场模型，给出 0–1800 s 内题定时刻、题定径向位置的结果，并形成逐秒完整结果。

问题 2：采用附录三随含水率和温度变化的物性，建立热湿耦合干燥模型，计算前 3 h 的温度和含水率分布以及完整过程结果。

问题 3：在问题二模型下，以药材内部所有位置的干基含水率均低于 0.15 为条件，确定临界时间并输出每 6 h 的径向含水率。

问题 4：进一步考虑附件二给出的半径收缩和附录四物性，建立移动域模型，确定全域达标时间，并辨析收缩与物性变化对结果的作用。

如图1所示，环境、几何、物性和终点判据共同汇入径向热湿守恒内核；中心对称、Robin 边界、真实表面闭合、环形有限体积和 BDF 积分由四问共享。问题一、二、四均从原始初态独立计算，只有问题三沿问题二的未舍入状态继续搜索全域事件。底部三类检验分别控制数值误差、输入与参数不确定性以及结构假设风险。

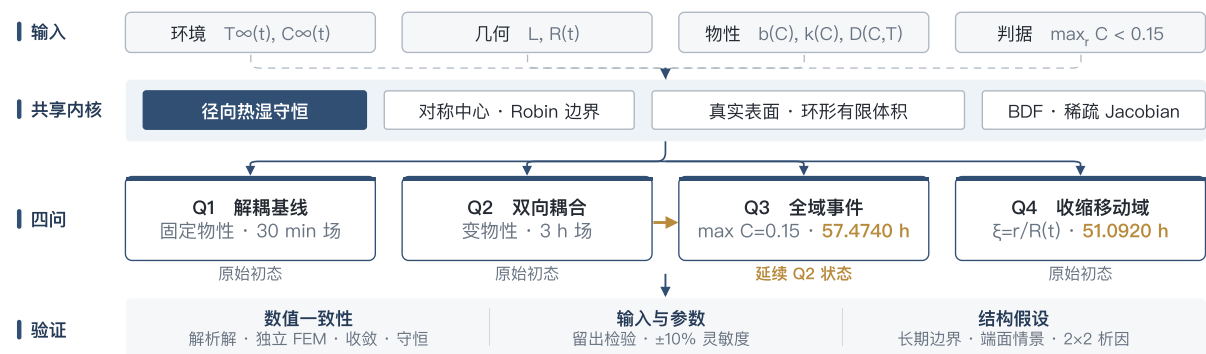


图 1 四问统一建模与验证框架

二、总体分析

2.1 数据处理与边界闭合

附件一给出的烘房温度与环境水分量具有明确时间顺序。为保留观测折点而不引入过冲，观测区间内采用分段线性插值；问题三、四超过 4 h 后，将 3–4 h（含端点）的观测均值作为平台值，即

$$T_{\text{air}} = 49.998934\text{ }^{\circ}\text{C}, \quad C_{\text{eq}} = 0.0499875.$$

其中 C_{eq} 表示药材表面一侧的等效平衡含水率。空气水分量与药材干基含水率的参照质量不同，题目又未给出解吸等温线，故主情景取 $C_{\text{eq}} = Y$ 作为边界闭合。第七、节除考察基准附近扰动外，还将 $C_{\text{eq}} = \beta Y$ 扩展到 $\beta = 0.5\text{--}2.8$ ，检验长期平衡含水率接近停机阈值时的终点变化。

附件二的半径观测在 0–72 h 内采用分段线性插值，问题四在 51.0920 h 即达到阈值，故无需外推半径。保形三次插值仅作为敏感性对照；它未在留出检验中表现出一致优势，因此不替换主方案 [4]。

2.2 尺度分析与模型选择

设有效体积热容 $b = \hat{\rho}c_p$ 、热扩散率 $\alpha = k/b$ ，以初始半径 R_0 为特征长度，定义

$$\tau_T = \frac{R_0^2}{\alpha}, \quad \tau_C = \frac{R_0^2}{D}, \quad Bi_T = \frac{hR_0}{k}, \quad Bi_C = \frac{h_m R_0}{D}.$$

三套物性的初始热扩散时间约为 39.48、46.03 和 84.93 min，而水分扩散时间约为 22.50、19.69 和 106.11 h；相应 Bi_T 和 Bi_C 均不满足小 Bi 数集总近似条件。因此，温度和含水率都保留径向分布。图2同时展示外部驱动与材料尺度：烘房环境在数小时内接近平台，但水分扩散和收缩延续更久，不能把环境稳定时刻当作药材干燥终点。

图2(c) 把观测支持与计算时域放在同一时间轴上：环境仅观测至 4 h，故问题三、四必须显式声明长期平台闭合；半径观测覆盖 72 h，超过问题四的 51.0920 h 终点，不需要几何外推。图2(d)(e) 又表明三套物性的水分时间尺度均显著长于热时间尺度，且初始 Bi_T 、 Bi_C 均远高于集总近似常用界限 0.1。这两条证据共同支持“保留径向分布、分别处理观测内插与长期延拓”的模型选择。

2.3 四问的递进关系

问题一中热学物性为常数，扩散率只依赖含水率，温度场和水分场可分别求解；问题二、三中 T 影响 D ， C 又影响 D 、 b 和 k ，形成物性系数介导的双向反馈；问题三关注最后一个未达标位置，事件函数取全域最大含水率；问题四中半径随时间改变，需要

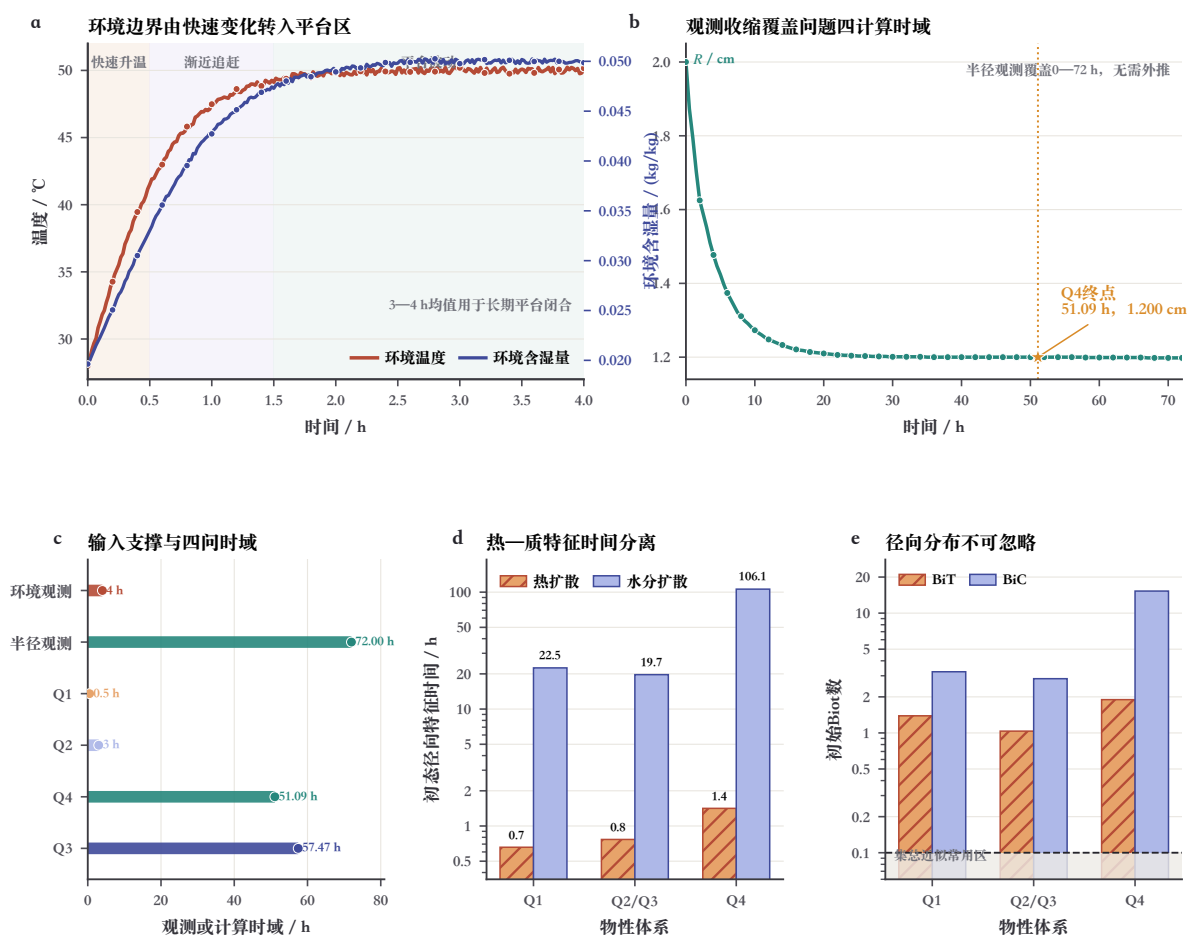


图 2 环境驱动与特征尺度

在材料参考坐标中写出守恒方程。四问的核心依次是早期边界层、变物性反馈、阈值事件定位和移动域守恒。

表1列出四问之间“继承什么、增加什么”的关系。除问题三续接问题二的同一轨迹外，其余各问均从题给原始初态重算，避免把不同物性模型的末态错误串联。

三、模型假设

在不改变题给经验物性的前提下，作如下假设并明确其适用范围：

- 1) 药材初始温度和干基含水率在内部均匀；材料轴对称，热物性和质扩散率在局部可视为各向同性标量。
- 2) 主模型描述几何中截面的径向过程，端面取绝热、不透湿边界。题面未说明端面是否暴露，因此该条件作为模型假设；附录另用二维轴对称暴露端面对照检验其影响。
- 3) 侧表面热交换满足牛顿冷却定律，水分交换满足线性 Robin 条件。问题二至四沿用题给 h 和 h_m ；主情景取 $C_{eq} = Y$ ，并通过宽范围比例系数 β 考察这一闭合对长期终点的影响。

表 1 四问建模递进

问题	模型新增因素	初态与时间范围	核心输出
一	固定域解耦热传导与水分扩散	原始初态, 0–30 min	题定时空温湿场
二	状态相关物性与双向热湿反馈	原始初态, 0–3 h	前三小时耦合响应
三	全域最大值事件与严格判停	续接问题二, 直至达标	临界时间及全程剖面
四	实测径向收缩与材料坐标	原始初态, 直至达标	移动域终点及因素分解标

- 4) 将题给经验密度 $\hat{\rho}(C)$ 仅用于构成有效体积热容 $b(C) = \hat{\rho}(C)c_p(C)$ 。水分方程按干固体质量守恒建立, 不把 $\hat{\rho}/(1+C)$ 同时强制解释为严格干固体密度。
- 5) 问题四仅发生径向仿射收缩, 长度保持 25 cm, 材料速度与网格速度一致; 半径 $R(t)$ 由附件观测指定, 而非由含水率反演。
- 6) 忽略辐射、蒸发潜热、水分携带焓、形变功和化学反应。因而热方程是题设参数下的有效显热模型, 其守恒残差不代表完整多相能量守恒。

图3给出主模型的几何、边界与问题四坐标映射。固定域以薄环 $V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2)L$ 为控制体, 用两侧面通量之差闭合局部守恒; 收缩域采用 $\xi = r/R(t) \in [0, 1]$, 将移动物理边界映射为固定材料边界。本文的“全域”特指所建一维径向域, 端面影响另作二维情景审查。

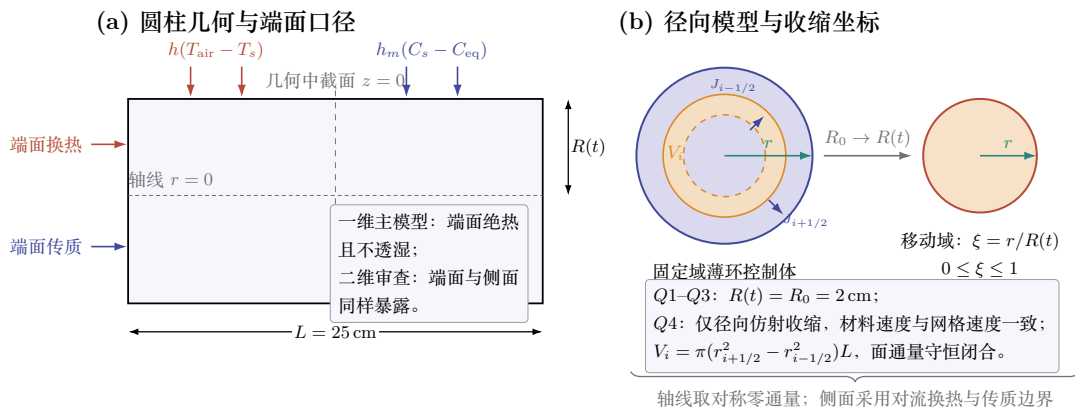


图 3 几何边界与收缩坐标

四、符号说明

本文主要符号见表2。摄氏温度用于场值输出，只有经验指数中的温度使用绝对温度 $T_K = T + 273.15$ 。

表 2 主要符号

符号	含义	单位
t	自烘干开始的物理时间	s
$r, R(t), L$	径向坐标、当前半径、圆柱长度	m
ξ	材料参考坐标 $r/R(t)$	1
T, T_K	摄氏温度与绝对温度	°C, K
C	药材干基含水率	kg/kg
Y, C_{eq}	环境水分量、药材侧等效边界含水率	kg/kg
$\hat{\rho}, c_p, b$	经验密度、比热容、有效体积热容	kg/m ³ , J/(kg·K), J/(m ³ ·K)
k, α	导热系数、热扩散率 k/b	W/(m·K), m ² /s
D	有效水分扩散率	m ² /s
h, h_m	对流换热系数、表面传质系数	W/(m ² ·K), m/s
β, S_p	平衡边界比例系数、无量纲局部灵敏度系数	1
s	单位当前体积内的干固体质量	kg/m ³
v, w	材料速度与移动网格速度	m/s
$\bar{C}, C_{max}$	干质量加权平均与全域最大含水率	kg/kg
t_*	$C_{max} = 0.15$ 的临界时刻	s 或 h

五、模型的建立与求解

5.1 统一的固定域热湿运输模型

5.1.1 建模思路与局部守恒

四问虽然在物性、时间范围和几何上逐步增加复杂性，但热量和水分都遵循同一条“控制体内积累量等于净流入量”的主线。为避免把干基含水率误作体积浓度，先在半径 r 处取厚度为 dr 的同心薄环。记径向向外为正方向，导热通量和相对于固体的水分通量分别为

$$q_T = -k(C) \frac{\partial T}{\partial r}, \quad q_C = -D(C, T) \frac{\partial C}{\partial r}. \quad (1)$$

薄环的外、内表面积分别为 $2\pi(r+dr)L$ 和 $2\pi rL$ 。将净流入量除以薄环体积并令 $dr \rightarrow 0$ ，可得

$$b(C) \frac{\partial T}{\partial t} = -\frac{1}{r} \frac{\partial(rq_T)}{\partial r}, \quad \frac{\partial C}{\partial t} = -\frac{1}{r} \frac{\partial(rq_C)}{\partial r}. \quad (2)$$

这里 $b(C) = \hat{\rho}(C)c_p(C)$ 作为题给经验关系下的有效体积热容，水分方程则按单位干固体质量归一化。两式的守恒测度不同，正是后续不能直接把 $\hat{\rho}C$ 写入水分积累项的原因。

5.1.2 控制方程

记圆柱初始半径为 $R_0 = 0.02$ m，长度为 $L = 0.25$ m。对固定几何问题，在忽略轴向变化后，圆柱径向热传导与干基水分扩散方程写为

$$b(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(rk(C) \frac{\partial T}{\partial r} \right), \quad (3)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(rD(C, T) \frac{\partial C}{\partial r} \right). \quad (4)$$

式(3)、(4)由式(1)代入式(2)得到。式(4)中的 C 是单位干药材质量所含水分，不是单位体积水浓度；固定域中干固体密度在归一化方程两侧约去。热方程写成 $b(C)T_t$ 而非 $[b(C)T]_t$ ，对应本文采用的有效显热闭合；变系数引起的收支修正将在模型检验中单独核验。

初始条件为

$$T(r, 0) = 28^\circ\text{C}, \quad C(r, 0) = 2.55. \quad (5)$$

中心满足对称条件，侧表面满足 Robin 边界：

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0, \quad (6)$$

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R_0} = h(T_s - T_{\text{air}}), \quad -D \left. \frac{\partial C}{\partial r} \right|_{r=R_0} = h_m(C_s - C_{\text{eq}}). \quad (7)$$

加热阶段 $T_s < T_{\text{air}}$ ，故式(7)的热通量为负，表示热量沿径向向内传递；排湿阶段 $C_s > C_{\text{eq}}$ ，水分向外通量为正，符号与物理方向一致。

5.1.3 真实表面值与边界闭合

题目要求给出 $r = R$ 处结果，而有限体积未知量位于最外层控制体中心 $r_N < R$ 。若直接用 T_N, C_N 代替表面值，会在 Robin 边界处引入半个网格宽度的系统偏差。记 $\delta_s = R - r_N$ ，将最后半单元扩散阻力与外部膜阻力串联，有

$$\frac{T_N - T_s}{\delta_s/k_{N,s}} = h(T_s - T_{\text{air}}), \quad (8)$$

$$\frac{1}{\delta_s} \int_{C_s}^{C_N} D(C, \bar{T}_{N,s}) dC = h_m(C_s - C_{\text{eq}}). \quad (9)$$

常物性时式(8)就是两段热阻的显式串联；变物性时， k 在半单元平均含水率处更新，式(9)保留 D 对 C 的变化，并与表面温度联立求解。程序使用区间保护的牛顿迭代，必要时退化为有界标量求根，因此得到的是满足边界方程的真实表面值，而非外推值。

5.1.4 环形有限体积离散

将径向区域划分为表面加密的 $N = 1280$ 个环形控制体，边界节点取

$$r_j = R_0 \left[1 - \left(1 - \frac{j}{N} \right)^2 \right], \quad j = 0, 1, \dots, N. \quad (10)$$

第 i 个控制体体积和内外侧面积分别为

$$V_i = \pi L(r_{i+1/2}^2 - r_{i-1/2}^2), \quad A_{i\pm 1/2} = 2\pi L r_{i\pm 1/2}.$$

对任意场 $\phi \in \{T, C\}$ ，令 $\Gamma = k$ 或 D ，内部面向外通量采用两侧半网格阻力串联：

$$q_{i+1/2} = \frac{\phi_i - \phi_{i+1}}{(r_{i+1/2} - r_i)/\Gamma_i + (r_{i+1} - r_{i+1/2})/\Gamma_{i+1}}. \quad (11)$$

于是半离散方程统一写成

$$M_i \frac{d\phi_i}{dt} = A_{i-1/2} q_{i-1/2} - A_{i+1/2} q_{i+1/2}, \quad M_i = \begin{cases} b_i V_i, & \phi = T, \\ V_i, & \phi = C. \end{cases} \quad (12)$$

中心面取 $q_{1/2} = 0$ ，外表面通量由式(7)给出。相邻控制体在公共面使用完全相同、符号相反的通量，求和后所有内部通量逐项抵消，因而离散层面保持局部与整体守恒。表面加密网格则把计算资源集中到温湿梯度最大的区域。

5.1.5 时间积分与结果重建

离散后得到 $2N$ 维非线性刚性常微分方程组。本文采用带解析稀疏 Jacobian 的变步长 BDF 方法 [5]，并按以下步骤求解：

- 1) 读取附件观测并构造分段线性环境函数；在每个观测折点前后分别积分，避免求解器跨越导数不连续点；
- 2) 在每次右端函数计算中，由当前 T_i, C_i 更新 b, k, D ，计算内部面通量和真实表面状态；
- 3) BDF 根据局部误差自适应选取内部步长，题定的 1 s 或 60 s 只作为稠密输出间隔；
- 4) 用中心偶对称二次式、内部区间保形三次插值和边界闭合值重建题定半径处结果；
- 5) 对问题三、四继续在每个积分片段内搜索全域事件，并在事件后独立推进以核验严格不等式。

主计算设置见表3。统一采用 $N = 1280$ 和二次表面加密；相对容差取 10^{-10} ，温度与含水率绝对容差分别为 10^{-11} 和 10^{-13} 。指定径向位置的场值采用保形分段三次重建，避免普通高阶插值在陡峭表面层产生过冲 [4]。

表 3 主计算设置

问题	积分区间	最大内部步长/s	输出间隔/s	初态来源
一	0–1800 s	1	1	原始均匀初态
二	0–3 h	2	1	原始均匀初态
三	3 h–阈值事件	30	60	问题二全精度末态
四	0–阈值事件	30	60	原始均匀初态

5.2 问题一：解耦预热模型

5.2.1 物性与模型特点

问题一采用附录二常热物性

$$\hat{\rho} = 820, \quad c_p = 2600, \quad k = 0.36,$$

以及含水率相关扩散率

$$D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) \text{ m}^2/\text{s}. \quad (13)$$

由于热方程系数不依赖 C ，且式(13)不依赖 T ，两场在本问中相互解耦。它们仍共享同一时变环境时间轴，但可分别积分。

5.2.2 求解过程与尺度判断

将常热物性代入式(3)，热扩散率为

$$\alpha = \frac{k}{\hat{\rho}c_p} = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}. \quad (14)$$

初态下 $D(C_0) = 4.9376 \times 10^{-9} \text{ m}^2/\text{s}$ 。在 $t = 1800 \text{ s}$ 时，以 $\ell \sim \sqrt{at}$ 估计，热扩散长度和水分扩散长度分别约为 1.743 cm 和 0.298 cm 。前者已接近半径 2 cm ，后者仅覆盖外层约 15% 的径向距离，预示温度剖面会整体抬升，而含水率会形成明显表面层。

具体计算时分别构造 N 维热方程和水分方程。热问题的系数矩阵只随网格和常物性确定；水分问题则在每个 BDF 内部步重新计算 $D(C_i)$ 。由于

$$\frac{dD}{dC} = D(C) \frac{0.89}{C^2} > 0, \quad (15)$$

表层失水会使当地扩散率下降，产生一定的自抑制作用。两场求解完成后在统一的 1 s 时间轴和 0.1 cm 径向网格上重建输出，保证温湿结果能够逐点对应。

5.2.3 结果与分析

题定温度和含水率结果分别见表4、表5。

表4 不同时刻的径向温度 ($^{\circ}\text{C}$)

时间/s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

图4给出完整时空演化。30 min 时，中心、表面温度分别达到 33.5753°C 和 36.7856°C ，横截面加权平均温度为 35.1679°C ；同期中心含水率仍为 2.5500 ，表面已降至 1.5102 ，加权平均为 2.2936 。初态估计的热扩散长度约 1.743 cm ，水分扩散长度仅约 0.298 cm ，与“热影响进入内部而显著失水集中于表层”的计算结果一致。

表 5 不同时刻的径向干基含水率 (kg/kg)

时间/s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

图4(a)(b) 从时空场展示传播范围, (c)(d) 用同一组时刻的径向剖面揭示梯度形状, (e)(f) 再把局部场压缩为体积平均响应和中心—表面差。30 min 时温差与含水率差分别达到 3.2103 °C 和 1.0398, 二者仍同步增大; 因此表面含水率快速下降并不意味着出现锐利的干燥前沿。中心值在四位小数下暂未变化, 也不等于中心水分在数学上绝对静止, 而是早期变化小于显示分辨率。

从加权平均量看, 30 min 时温度已由 28 °C 升至 35.1679 °C, 完成了相对于同期环境温度的大部分响应; 平均含水率仍为 2.2936, 保留初始值的约 89.9%。这组对比从整体量和局部剖面两方面验证了尺度分析, 不依赖某一个抽样半径的偶然结果。

5.3 问题二: 变物性热湿耦合模型

5.3.1 耦合关系

问题二使用附录三物性:

$$\hat{\rho}(C) = 650 + 128C, \quad (16)$$

$$c_p(C) = 1450 + 2736 \frac{C}{1+C}, \quad (17)$$

$$k(C) = 0.21 + 0.38 \frac{C}{1+C}, \quad (18)$$

$$D(C, T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left[-\frac{3850}{T + 273.15}\right]. \quad (19)$$

温度升高使式(19)增大, 含水率降低则使其减小; 与此同时, C 还通过 $b(C)$ 和 $k(C)$ 改变温度响应, 由此形成物性系数介导的双向反馈。数值求解时在每次残差计算中更新全部局部系数, 并保留散度算子内部的变系数 $\nabla \cdot (k \nabla T)$ 。

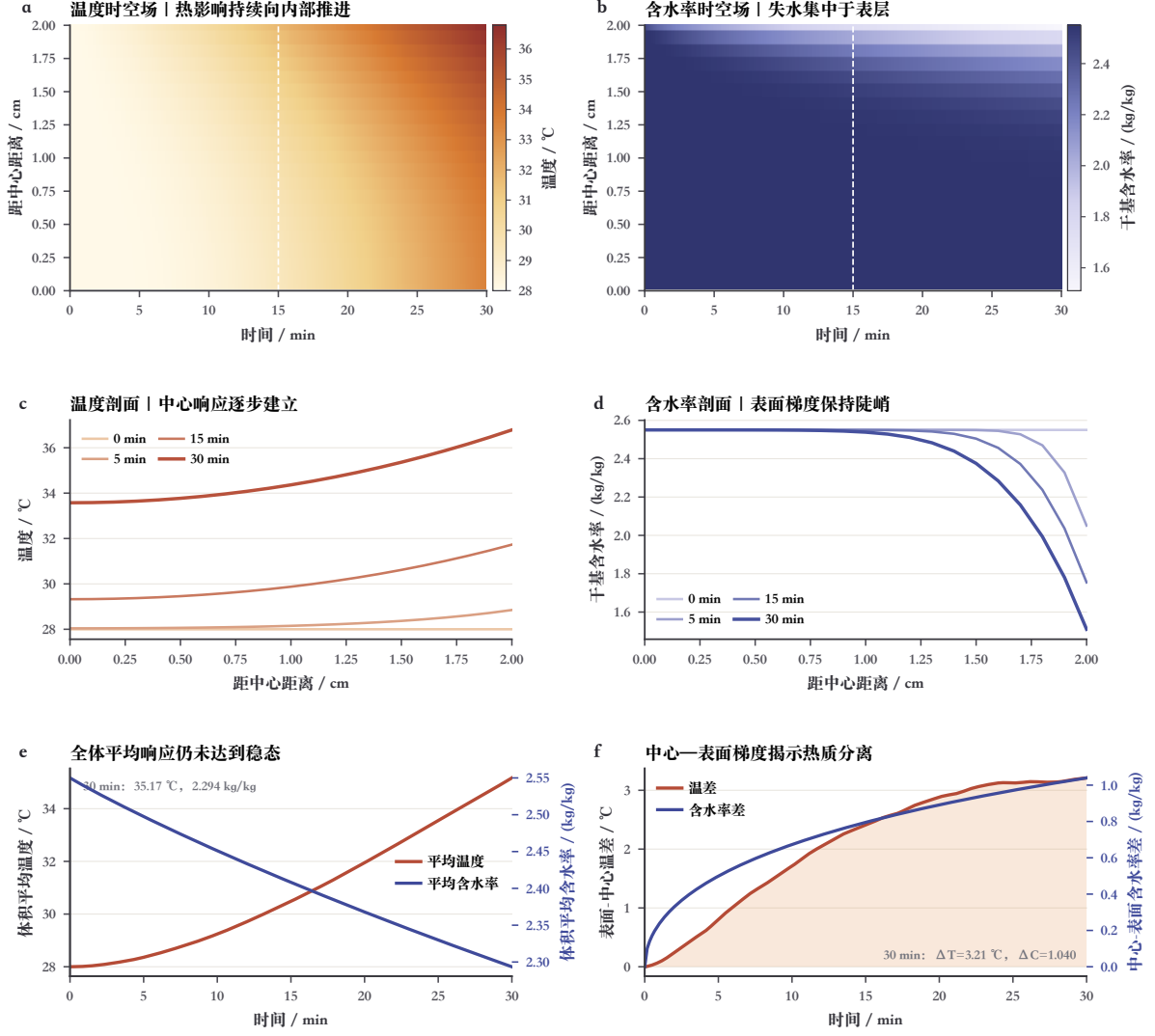


图4 问题一温湿场演化

由扩散率经验式直接求偏导，有

$$\frac{\partial \ln D}{\partial T} = \frac{3850}{(T + 273.15)^2} > 0, \quad \frac{\partial \ln D}{\partial C} = \frac{0.45}{C^2} > 0. \quad (20)$$

因此升温促进内部扩散，而失水通过降低 C 抑制扩散。另一方面， $\hat{\rho}(C)$ 、 $c_p(C)$ 和 $k(C)$ 均随 C 增加；脱水会同时减小体积热容和导热系数，前者倾向于加快局部升温，后者倾向于增强温度梯度。两类作用方向不同，需要联立求解后作定量判断。

相对于初态 (C_0, T_0) ，扩散率的对数变化可拆为

$$\ln \frac{D(C, T)}{D(C_0, T_0)} = 3850 \left(\frac{1}{T_0 + 273.15} - \frac{1}{T + 273.15} \right) + 0.45 \left(\frac{1}{C_0} - \frac{1}{C} \right). \quad (21)$$

该式便于辨别升温促进与脱水抑制的相反作用。

5.3.2 耦合求解与机制辨识

按每个控制体交错存储状态向量

$$\mathbf{y} = (T_1, C_1, T_2, C_2, \dots, T_N, C_N)^T, \quad (22)$$

将式(12)写成 $\dot{\mathbf{y}} = \mathbf{F}(t, \mathbf{y})$ 。内部热通量同时依赖相邻 T 和 C ，水分通量则依赖相邻 T 、 C 以及 $D(C, T)$ ；表面 T_s, C_s 又通过式(8)、(9)相互联系。因此 Jacobian 呈块三对角结构。程序解析计算各通量对相邻状态的偏导并以稀疏矩阵交给 BDF 方法，避免在 $N = 1280$ 时使用稠密数值差分。

本问的计算流程为：从原始均匀初态出发，在 0–3 h 的每个环境折点处分段积分；每个内部步更新局部物性与表面状态；完成后将全精度末态连同网格、环境口径和源文件指纹保存，供问题三无损续算。为识别耦合机制，另设置“冻结 D 的温度因子”情景：温场仍按原模型求解，但扩散率中的温度固定为初温。该对照只删除一条反馈路径，其余条件完全相同。

5.3.3 前三小时结果

问题二从原始初态重新计算，并非继承问题一的 1800 s 状态。题定结果汇总于表6，其中 (a)(b) 分别给出相同时刻、相同径向位置的温度与干基含水率，便于逐项对照。

表 6 三小时内药材温湿分布

(a) 温度 (°C)						(b) 干基含水率 (kg/kg)					
时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm	时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
0.5	32.1892	32.3821	32.9659	33.9608	35.4130	0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1.0	40.3816	40.5536	41.0605	41.8782	42.9976	1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	45.8468	45.9348	46.1930	46.6051	47.1400	1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	48.4502	48.4881	48.5984	48.7735	49.0033	2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	49.4670	49.4792	49.5136	49.5653	49.6609	2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	49.8495	49.8553	49.8746	49.9101	49.9664	3.0	1.7662	1.7165	1.5702	1.3333	1.0081

3 h 时，中心、表面温度为 49.8495 °C 和 49.9664 °C，仅相差 0.1169 °C；中心、表面含水率却仍为 1.7662 和 1.0081，差值达到 0.7581。图5说明温度比含水率更快接近平台，故不能用“温度接近 50 °C”代替干燥完成判据。

将式(21)代入 3 h 状态，中心的温度贡献为 +0.8648，含水率贡献为 −0.0783；表面分别为 +0.8691 和 −0.2699。此时升温促进仍占上风，但表面脱水抑制更强。若只冻结 D 中的温度因子而仍求解温场，3 h 中心含水率会由 1.7662 增至 2.1933，表明忽略升温

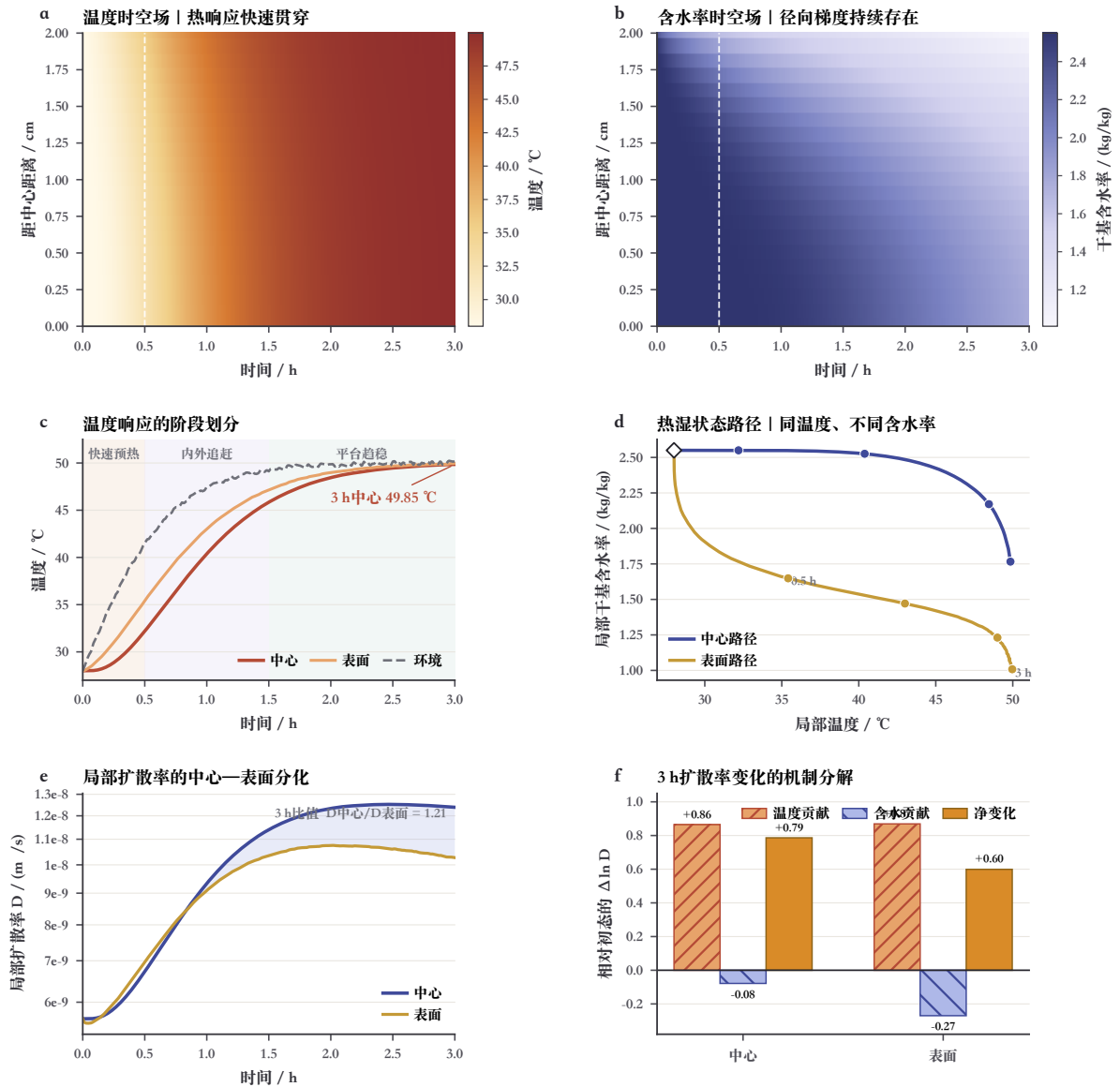


图5 问题二耦合响应

对扩散的作用会明显低估前三小时失水。该对照是模型内机制消融，不是实验因果识别。

图5(d) 把中心和表面的 (T, C) 状态路径直接投影到同一平面，可见二者在接近相同温度后仍保持不同含水率；(e) 进一步显示中心扩散率与表面扩散率的分化；(f) 则将 $\Delta \ln D$ 拆成温度项、含水项和净变化。这三个面板把“场的差异”与“物性公式中的竞争机制”连接起来，而不是只给出两张结果热图。

进一步比较剖面可见，温度的中心—表面差已由早期的数摄氏度收缩到 $0.1169\text{ }^{\circ}\text{C}$ ，含水率差却仍为 0.7581 。这是因为热扩散时间以分钟计，而水分扩散时间以小时计；同时表层含水率下降又使其局部 D 减小，延缓内部水分向表面的补给。故“温场近似均匀”和“水分场近似均匀”在本问题中属于两个完全不同的阶段。

5.4 问题三：全域阈值事件

5.4.1 事件定义与严格不等式

问题三与问题二使用同一物性、同一初态和同一物理时钟。为避免只检查少数表格位置，定义

$$g(t) = \max_{0 \leq r \leq R_0} C(r, t) - 0.15, \quad t_* = \inf\{t : g(t) \leq 0\}. \quad (23)$$

最大值搜索包含中心、全部控制体、真实表面以及空间重建。另构造中心含水率约 0.10、内部 $r \approx 1.1 \text{ cm}$ 处峰值约 0.298 的反例，中心判据会误报达标，而式(23)可正确拒绝。因此，即使本次实际轨迹的最湿点最终位于中心，也不将“中心恒为最湿”预设到算法中。

连续模型在 t_* 满足等号，严格条件 $C_{\max} < 0.15$ 通常在 t_* 后的开区间成立，不存在被取到的“最早严格时刻”。因此本文同时报告等号临界时刻和向上取整后重新核验的整数秒。

5.4.2 全域搜索与事件定位算法

有限体积状态位于单元中心，还需把中心轴和真实表面纳入最大值搜索。中心偶对称条件给出局部形式 $C(r) = a + br^2$ ，由前两个单元中心值可恢复轴心值

$$C(0, t) = \frac{r_2^2 C_1 - r_1^2 C_2}{r_2^2 - r_1^2}. \quad (24)$$

内部区间采用保形三次重建，其区间值不超过相邻节点包络；第一半单元的偶二次式关于 r^2 单调。因此事件函数只需在

$$\mathcal{S}(t) = \{C(0, t), C_1(t), \dots, C_N(t), C_s(t)\} \quad (25)$$

上取最大值，即可覆盖重建后的完整径向域，而不是只检查题目表格中的 21 个位置。

事件定位按以下流程执行：首先读取问题二 3 h 时的未舍入状态，并验证此时 $g(t) > 0$ ；随后以 30 s 为最大内部步长分段续算，在每个 BDF 片段上调用稠密输出计算 $g(t)$ ；当检测到负方向过零时，由连续插值自动细化根的位置；最后分别回代 $t_* - 1 \text{ s}$ 、 t_* 和 $t_* + 1 \text{ s}$ ，并从 t_* 独立推进到下一整数秒。程序还在所有接受步与步中点检查 $C_{\max}(t)$ 的单调性，避免由于状态重建或求根切换产生伪事件。

5.4.3 终点与后期机制

求得

$$t_* = 206906.491182 \text{ s} = 57.4740253 \text{ h}, \quad (26)$$

按题定格式为 57.4740 h；首个经核验严格达标的整数秒为 206907 s，即 57 h 28 min 27 s。题定含水率结果见表7。

表 7 全过程径向干基含水率 (kg/kg)

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
6.0000	1.0170	0.9881	0.9015	0.7550	0.5328
12.0000	0.4561	0.4432	0.4031	0.3297	0.1642
18.0000	0.2988	0.2916	0.2687	0.2253	0.0845
24.0000	0.2378	0.2327	0.2167	0.1855	0.0665
30.0000	0.2058	0.2018	0.1892	0.1641	0.0598
36.0000	0.1859	0.1825	0.1718	0.1504	0.0566
42.0000	0.1721	0.1691	0.1597	0.1407	0.0548
48.0000	0.1618	0.1592	0.1507	0.1334	0.0537
54.0000	0.1539	0.1514	0.1436	0.1277	0.0530
57.4740	0.1500	0.1477	0.1402	0.1249	0.0526

图6显示，终点中心值为 0.1500，表面值已降至 0.0526，加权平均含水率为 0.1243。临界前后 1 s 的 $g(t)$ 分别约为 $+2.913 \times 10^{-7}$ 和 -2.913×10^{-7} ，证明事件发生的是向下穿越而非采样偶然命中。

图6(b)量化了判据选择造成的偏差：表面值和体积平均值分别约在 12.55 h 和 36.03 h 达到 0.15，比全域判据提前 44.92 h 和 21.45 h；(d) 显示截面达标比例虽不断增大，但必须到最后湿点达标时才达到 100%。在临界点，升温对 $\ln D$ 的累计贡献约为 +0.8703，含水率下降的贡献在中心和表面分别约为 -2.8235 和 -8.3710 。中心扩散率约 $8.00 \times 10^{-10} \text{ m}^2/\text{s}$ ，表面仅约 $3.12 \times 10^{-12} \text{ m}^2/\text{s}$ ，相差约 257 倍。低含水表层的扩散率下降与显著内部梯度共同解释了后期减速；但外部传质系数仍影响终点，故不将全过程概括为“完全由内部扩散控制”。

按全域判据，终点由最后一个达到阈值的位置控制。虽然本次主轨迹的最大值在后期位于中心，但这是求解结果而非先验条件。终点平均含水率 0.1243 明显低于阈值，表面更低至 0.0526；若用平均值代替最大值，必然会提前停机。反之，用题定四位小数直接搜索也会把舍入误差带入事件时间，因此本文始终在未舍入状态上定位事件，只在最终表格中按格式舍入。

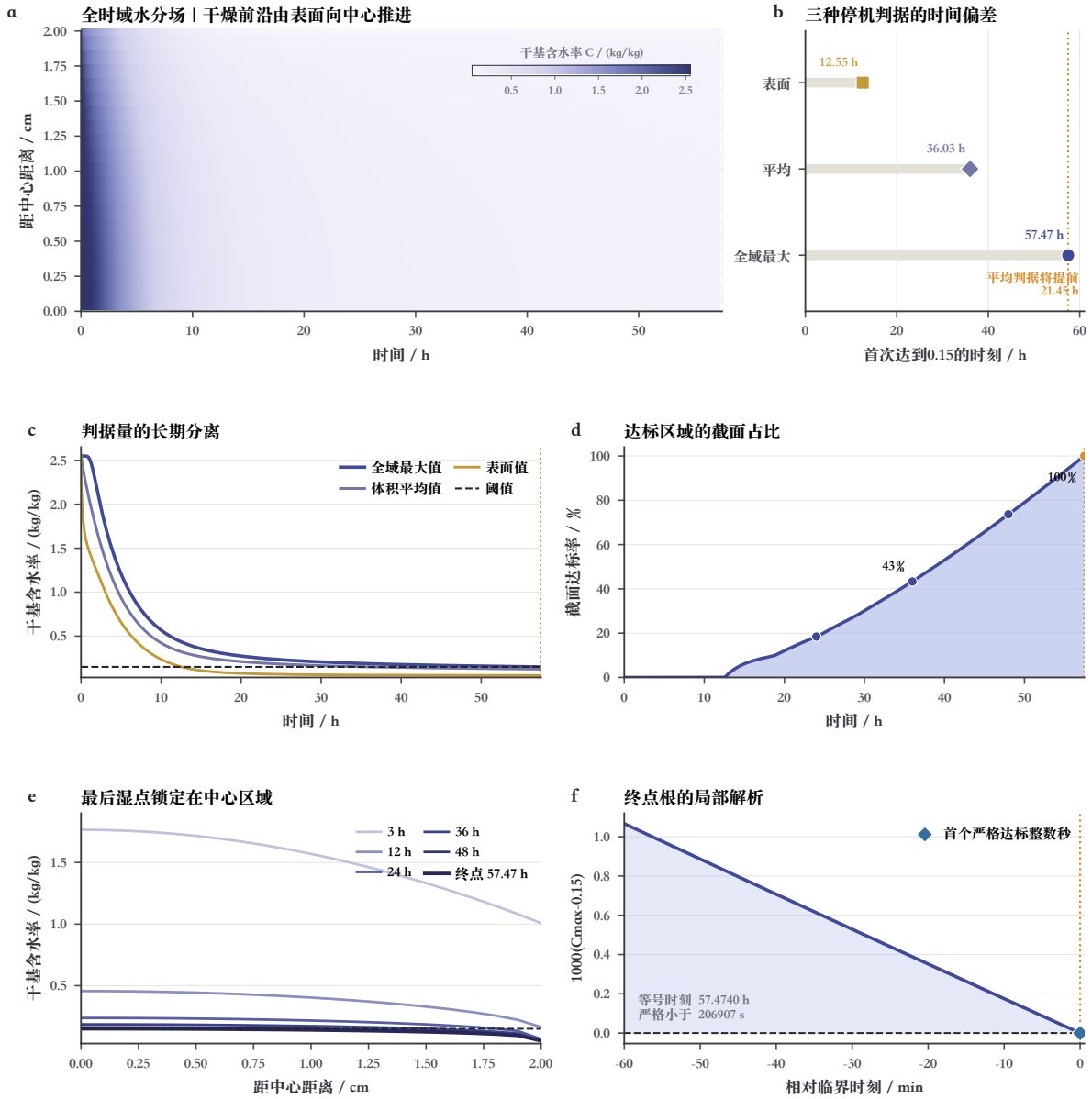


图 6 问题三全域干燥终点

5.4.4 第三问交叉验证

为避免同一离散框架内部自洽被误认为交叉验证，另以不同空间离散、不同时间推进和独立程序复算临界事件，结果见表8。其中独立程序采用 P1 有限元弱式、一致质量矩阵、自编 BDF2 与 Picard 迭代，并从独立求得的问题二节点末态继续；它不调用主程序的有限体积面通量、Jacobian、表面重建或事件函数。

五种计算的小时值均舍入为 57.4740 h。主解网格由 1280 加密到 2560 时，临界时间改变 -0.075956 s ；接近二阶收敛估计，主网格终点的保守数值尺度约为 0.1 s 。独立有限元与主解相差 0.002099 s ，公共保存时刻的最大温差和含水率差分别为 $3.028 \times 10^{-5} \text{ }^{\circ}\text{C}$ 与 3.308×10^{-7} ，说明不同离散路线得到一致结果，但这一毫秒级差值不作为误差上界。

表 8 问题三临界时间交叉验证

证据	空间与时间离散	t_*/s	相对主解 /s
主解	FVM $N = 1280$, BDF 30 s	206906.491182	—
全新复算	同主解, 新缓存重新积分	206906.491182	0
积分器替换	FVM $N = 1280$, Radau 30 s	206906.491402	+0.000220
空间加密	FVM $N = 2560$, BDF 30 s	206906.415227	-0.075956
独立程序	P1 FEM $N = 2560$, BDF2 3.75 s	206906.493281	+0.002099

事件邻域回代给出

$$g(t_* - 1 \text{ s}) = 2.913 \times 10^{-7}, \quad g(t_*) = 0, \quad g(206907 \text{ s}) = -1.482 \times 10^{-7},$$

且所有接受步与步中点上的 $C_{\max}$ 均未出现回升。水分累计守恒相对残差为 6.49×10^{-10} 。综合网格、积分器和独立离散结果, 第三问在题定四位小数下稳定; 实际工艺精度仍取决于物性和边界参数。

5.5 问题四: 收缩域材料坐标模型

5.5.1 由干固体守恒得到移动域方程

设 s 为单位当前体积内的干固体质量, v 为材料速度, 相对于干固体的水分通量为 $J_w = -sD\nabla C$ 。干固体与水分守恒分别为

$$\frac{\partial s}{\partial t} + \nabla \cdot (sv) = 0, \quad \frac{\partial (sC)}{\partial t} + \nabla \cdot (sCv + J_w) = 0. \quad (27)$$

第二式减去 C 乘第一式, 得

$$s \frac{DC}{Dt} = -\nabla \cdot J_w, \quad \frac{D}{Dt} = \frac{\partial}{\partial t} + v \cdot \nabla. \quad (28)$$

仿射径向收缩且长度不变时, $v_r = (\dot{R}/R)r$, 故

$$\nabla \cdot v = \frac{1}{r} \frac{\partial (rv_r)}{\partial r} = 2 \frac{\dot{R}}{R}, \quad \frac{Ds}{Dt} = -s \nabla \cdot v. \quad (29)$$

若初始干固体密度 s_0 空间均匀, 则

$$s(t) = s_0 \left(\frac{R_0}{R(t)} \right)^2. \quad (30)$$

同一时刻的 s 与 r 无关, 因而 $J_w = -sD\nabla C$ 中的 s 可从空间散度中提出, 并与式(28)左端约去。这是参考域水分方程不含额外 ∇s 项的条件。

令 $\xi = r/R(t)$, $u(\xi, t) = C(R(t)\xi, t)$, $\theta(\xi, t) = T(R(t)\xi, t)$ 。径向仿射收缩使 $v = (\dot{R}/R)r$, 网格速度 $w = \dot{R}\xi$ 与材料速度一致, 故材料导数在固定 ξ 处就是 u_t 。这一点可由链式法则写为

$$\left. \frac{\partial u}{\partial t} \right|_{\xi} = \frac{\partial C}{\partial t} + \dot{R}\xi \frac{\partial C}{\partial r} = \frac{DC}{Dt}, \quad \frac{\partial C}{\partial r} = \frac{1}{R} \frac{\partial u}{\partial \xi}. \quad (31)$$

式(31)表明, Euler 坐标中的对流项并未被忽略, 而是由于网格跟随材料运动而包含在固定 ξ 的时间导数中。再将 $r = R\xi$ 代入径向散度算子, 得到参考域方程

$$\frac{\partial u}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi D(u, \theta) \frac{\partial u}{\partial \xi} \right), \quad (32)$$

$$b(u) \frac{\partial \theta}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi k(u) \frac{\partial \theta}{\partial \xi} \right). \quad (33)$$

这与移动网格/ALE 思想相符 [6]。式(32)依赖材料运动 $v = w$ 、长度不变和初始 s_0 空间均匀; 干基含水率的压缩效应已由干固体连续方程处理。式(33)则是同一材料运动下采用的有效显热闭合, 并非由于固体守恒自动推出的完整能量方程。

若误把 C 当作单位当前体积的含水量, 坐标变换后会出现额外稀释或浓缩项, 并导致“关闭扩散和传质后 C 仍随收缩变化”的非物理解。本文以这一极端情景作为程序单元测试: 当 $D = h_m = 0$ 时, 任意非均匀干基含水率剖面在材料坐标中均保持不变, 从计算层面验证了式(32)的守恒口径。

表面条件变为

$$-\frac{k}{R} \theta_{\xi}(1, t) = h(\theta_s - T_{\text{air}}), \quad -\frac{D}{R} u_{\xi}(1, t) = h_m(u_s - C_{\text{eq}}). \quad (34)$$

按参考干质量测度, 平均含水率及其累计守恒式为

$$\bar{C}(t) = 2 \int_0^1 \xi u(\xi, t) d\xi, \quad (35)$$

$$E_C(t) = \bar{C}(t) - C_0 + \int_0^t \frac{2h_m}{R(\tau)} [u_s(\tau) - C_{\text{eq}}(\tau)] d\tau = 0. \quad (36)$$

问题四物性为

$$\hat{\rho}(u) = 760 + 90u, \quad (37)$$

$$c_p(u) = 1850 + 2150 \frac{u}{1+u}, \quad (38)$$

$$k(u) = 0.12 + 0.20 \frac{u}{1+u}, \quad (39)$$

$$D(u, \theta) = 4.2 \times 10^{-4} \exp \left(-\frac{0.30}{u} \right) \exp \left[-\frac{3850}{\theta + 273.15} \right]. \quad (40)$$

5.5.2 参考域离散与半径驱动

在参考网格中仍使用第12式的环形有限体积结构。令 $s_R(t) = R(t)/R_0$ ，则内部扩散算子整体乘以 s_R^{-2} ，而写回参考坐标的 Robin 系数变为 $s_R h$ 和 $s_R h_m$ 。这样可复用固定域通量程序，同时严格对应物理表面的 h, h_m ，并避免在代码中显式计算噪声较大的 $\dot{R}(t)$ 。

附件半径在 0–72 h 内分段线性插值。每到半径观测折点，求解器与环境折点一样分段重启；若积分要求超过 72 h 则主动报错，防止无标记外推。每个 60 s 输出时刻，先在材料坐标中重建 $u(\xi, t)$ ，再用 $r = R(t)\xi$ 映射到物理坐标。固定物理位置 r_j 只有在 $r_j \leq R(t)$ 时才有材料场值，否则保持空白。这一规则同时用于工作簿和图7的白色域外遮罩。

5.5.3 收缩域结果

问题四从原始初态独立计算，所得临界时刻为

$$t_* = 183931.288720 \text{ s} = 51.0920246 \text{ h},$$

(41)

按题定格式为 51.0920 h；此时 $R = 1.2000 \text{ cm}$ ，首个严格达标整数秒为 183932 s。题定输出见表9。当固定物理位置已位于收缩后的药材外部时，表中留空；空白不表示含水率为零，动态“药材表面”列始终对应 $r = R(t)$ 。

表 9 收缩药材干基含水率 (kg/kg)

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm	药材表面
6.0000	1.7196	1.5378	1.0226			0.4207
12.0000	0.7377	0.6546	0.4083			0.1669
18.0000	0.4088	0.3687	0.2397			0.0892
24.0000	0.2851	0.2611	0.1790			0.0673
30.0000	0.2263	0.2094	0.1493			0.0595
36.0000	0.1928	0.1797	0.1317			0.0560
42.0000	0.1712	0.1604	0.1200			0.0541
48.0000	0.1561	0.1468	0.1116			0.0530
51.0920	0.1500	0.1413	0.1081			0.0526

图7中白色区域同样表示域外。终点中心、表面和干质量加权平均含水率分别为0.1500、0.0526和0.1184。几何因子 $1/R^2$ 随半径减小而增大，且面积体积比 $2/R$ 增大；当 $R/R_0 = 0.6$ 时，两者分别相对初始放大2.7778倍和1.6667倍。不过，实际扩散率同时随状态变化，故这些倍率不能直接解释成同倍数的干燥速率。

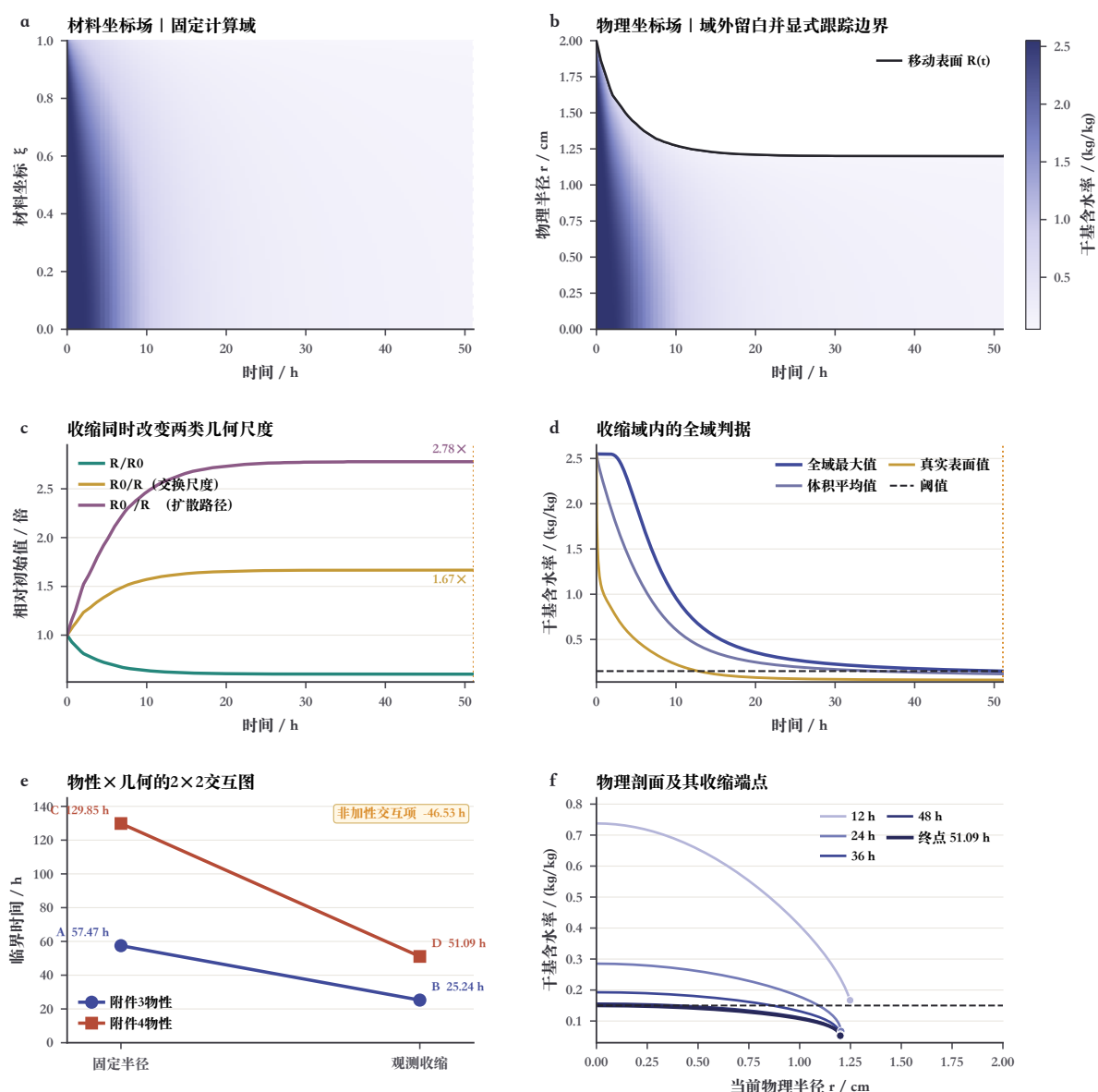


图7 问题四收缩域效应

图7(a)(b) 分别在材料坐标和物理坐标中呈现同一水分场，后者以移动边界和白色遮罩区分药材内外；(c) 同步给出 R/R_0 、 R_0/R 与 R_0^2/R^2 ，分别对应交换尺度和扩散路径尺度；(e) 以四组条件情景显示几何影响随物性背景改变。

5.5.4 统一半径轨迹下的条件对照

问题三和问题四同时改变了几何和物性。为辨别两类改变的条件影响，令 A 为附录三物性与固定半径，B 为附录三物性与实测收缩，C 为附录四物性与固定半径，D 为附录四物性与实测收缩，四组均从相同初态计算，结果见表10。B、D 使用同一条附件半径轨迹，因此这里比较的是给定外生几何驱动下的数值对照。

表 10 几何与物性对照

场景	物性	几何	临界时间/h	终点半径/cm
A	附录三	固定	57.4740	2.000
B	附录三	收缩	25.2440	1.203
C	附录四	固定	129.8484	2.000
D	附录四	收缩	51.0920	1.200

以 A 为参考，几何与物性的简单效应及交互对比分别为

$$\Delta_{\text{geom|III}} = t_B - t_A = -32.2301 \text{ h}, \quad (42)$$

$$\Delta_{\text{prop|fixed}} = t_C - t_A = +72.3744 \text{ h}, \quad (43)$$

$$\Delta_{\text{int}} = t_D - t_B - t_C + t_A = -46.5263 \text{ h}. \quad (44)$$

三项之和为 $t_D - t_A = -6.3820 \text{ h}$ 。几何的另一简单效应为 $t_D - t_C = -78.7564 \text{ h}$ ，物性的另一简单效应为 $t_D - t_B = +25.8481 \text{ h}$ 。几何和物性的影响均随另一因素的背景而改变， -46.5263 h 正是两组简单效应之差。

附件半径是题给输入，并未随情景物性重新求解，所以该对照描述当前模型中的条件差异。问题三、四的 6.3820 h 净差同时含有物性、几何及二者交互，并非单独的收缩贡献。

还需指出，若把问题四的经验密度同时严格解释为湿药材总密度，并要求长度固定、干固体质量守恒，则可推出 $R \geq 1.2112 \text{ cm}$ ；附件半径后期低于该值。本文因此采用“有效热学系数 + 独立干质量守恒”的解释，而不声称同时满足严格混合物密度关系。

六、模型检验与误差分析

验证围绕输入处理、方程口径、数值离散、独立实现和结构情景展开。题目未提供药材内部温湿实测值、参数分布或重复实验，因此本节评价数值一致性，并将物理预测能力单独讨论。

6.1 边界、量纲与极端情景检查

式(3)两侧单位均为 W/m^3 ，式(4)两侧均为 s^{-1} ；表面热通量和水分归一化通量分别为 W/m^2 与 m/s 。问题四中 D/R^2 具有 s^{-1} 量纲，材料坐标变换前后保持一致。

程序通过五项极端测试：常场绝热时右端为零；关闭扩散和传质后，非均匀干基含水率随材料收缩保持不变；令 $R(t)$ 为常数时，问题四退化为固定域离散式；计算到 72 h 即停止调用半径插值；人造非中心湿峰可被全域事件识别。这些结果用于检查符号、坐标和判停逻辑。

6.2 网格、时间步、守恒与独立求解

表11汇总主解与独立 P1 有限元的公共点差异。独立程序使用一致质量矩阵、Gauss 积分、节点 Robin 边界和自编 BDF2，不调用主有限体积程序的面通量、表面恢复或事件函数。问题一热场另与 512 项 Fourier–Bessel 解析级数比较，全时域最大差为 $1.285 \times 10^{-6} \text{ } ^\circ\text{C}$ 。

表 11 独立求解对照

对象	最大温差/ $^\circ\text{C}$	最大含水率差	抽检范围
问题一	1.801×10^{-6}	1.321×10^{-6}	10 个稠密剖面
问题二	9.161×10^{-6}	3.222×10^{-6}	6 个题定时刻
问题三	3.028×10^{-5}	3.308×10^{-7}	108 个保存时刻
问题四	1.093×10^{-5}	6.030×10^{-6}	110 个公共时刻

固定域的归一化水分累计残差为

$$E_C = \bar{C} - C_0 + \int_0^t \frac{2h_m}{R_0} (C_s - C_{\text{eq}}) \text{d}\tau. \quad (45)$$

当 $b(C)$ 变化时，以 $H = \overline{b(C)(T - T_0)}$ 构造的有效热收支还需包含 $b'(C)(T - T_0)C_t$ 修正项。四问水分与修正热学收支的最大归一化残差均小于 1.1×10^{-9} ，表明离散轨迹与所建方程一致。潜热等未建模项的影响列入模型局限。

图8(a)–(c) 显示相邻网格差总体呈近二阶下降；(d) 把最大步长、容差与 Radau 积分器的终点差异放到同一对数尺度；(e)(f) 分别给出独立有限元和累计守恒两条异源证据。问题三、四的 1280 与 2560 网格临界时间差分别为 0.07596 s 和 0.05797 s；时间步上限 30 s 减半后的变化为 0.000755 s 和 0.000462 s。接近二阶关系估计，主网格终点的数值尺度约为 0.1013 s 和 0.0773 s，本文据此采用 0.1 s 量级作为保守表述。

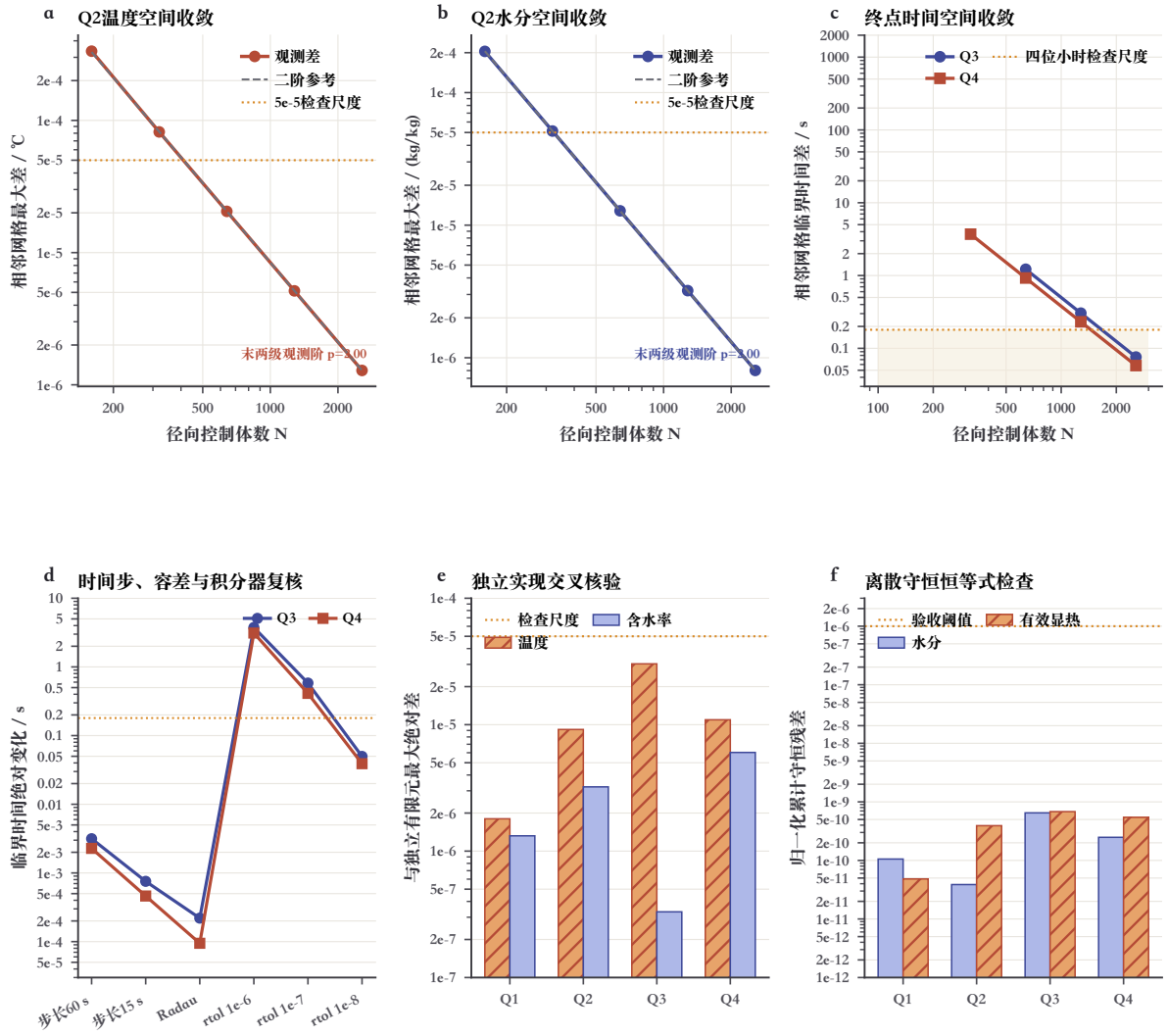


图8 数值收敛与独立验证

临界附近 $g(t)$ 的斜率分别约为

$$-2.913 \times 10^{-7} \quad \text{和} \quad -5.135 \times 10^{-7} \text{ (kg/kg)/s.}$$

由 $\delta t \approx \delta C/|g'|$ 可知，含水率显示误差 5×10^{-5} 可对应问题三约 172 s、问题四约 97 s 的时间尺度。因此，表格小数位服务于结果复核，终点的工艺精度应由实验和参数不确定性确定。

6.3 输入留出与工作簿回读

对附件观测作隔点留出：线性插值的环境温度、环境水分量与半径 RMSE 分别为 0.162°C 、 1.80×10^{-4} 和 $7.20 \times 10^{-4} \text{ cm}$ ，均优于“沿用上一观测值”的简单基线；PCHIP 未表现出一致优势。该检验支持观测区间内的线性插值，4 h 后的环境平台由第七、节另作情景分析。详细对比见附录图10。

四份结果工作簿共逐项核验 8 882 410 个数值单元格和问题四 23 165 个域外空白格, 显示值均与未舍入计算缓存的四舍五入结果一致, 时间、表头和域外空白也通过回读检查。

6.4 端面结构假设审查

端面条件的二维轴对称配对审查见附录图11。若令端面与侧面同样暴露, 问题二 0–3 h 中截面温度最大差约 0.0017 °C, 问题三终点提前约 7–8 s, 问题四约提前 0.0284 s; 同时整根平均含水率和端部局部场的变化明显大于中截面差异。因此, 一维结果对应端面封闭条件下的中截面过程; 整根平均量对端面开放程度更敏感。其他参数和结构情景见第七、节。

6.5 精度说明

在既定一维几何、等效边界闭合和题给物性下, 网格加密、时间步缩小、独立有限元与累计收支共同支持当前数值解, 问题三、四的临界时间在题定格式下分别为 57.4740 h 和 51.0920 h。数值离散的终点尺度约为 0.1 s, 实际预测误差主要由物性、边界闭合和实验条件决定。

七、灵敏度分析与稳健性讨论

本文从局部参数和边界闭合两个层面考察终点变化。局部分析选取换热系数 h 、传质系数 h_m 、扩散率整体尺度 D 和边界映射系数 β , 在其余条件不变时分别施加 $\pm 10\%$ 扰动; 其中 D 表示完整经验扩散率的乘数, $C_{eq} = \beta Y$ 。由于题目未给出参数分布, 以下结果均为确定性情景。

7.1 指标定义与试验设计

以临界时间 t_* 为响应量, 定义参数 p 的有符号无量纲局部灵敏度系数

$$S_p = \left. \frac{\partial \ln t_*}{\partial \ln p} \right|_{p_0} \approx \frac{t_*(1.1p_0) - t_*(0.9p_0)}{0.2t_*(p_0)}. \quad (46)$$

$S_p < 0$ 表示增大该参数会缩短干燥时间, $|S_p|$ 越大表示局部影响越强。式(46)使用对称差分, 可同时利用正、负扰动并降低单侧差分偏差。每个扰动场景均从相应原始初态完整重算, 重新搜索全域阈值事件, 而不是在基准终点附近线性外推。

表12给出两问的重算结果。问题三、四的基准时间分别为 57.4740 h 和 51.0920 h。

7.2 边界闭合的宽情景检验

$\pm 10\%$ 扰动只反映 $\beta = 1$ 附近的局部斜率, 对空气水分量到药材平衡含水率的定义差异覆盖不足。为此, 将 β 扩展到 0.5、2.0 和 2.8, 沿用相同环境、物性和事件判据完

表 12 临界时间局部灵敏度

问题	参数	扰动含义	$t_*(-10\%)/h$	$t_*(p_0)/h$	$t_*(+10\%)/h$	S_p
问题三	h	换热强度	57.4828	57.4740	57.4669	-0.0014
	h_m	表面传质	58.2099	57.4740	56.8967	-0.1142
	D	内部扩散	63.1322	57.4740	52.8666	-0.8931
	β	平衡边界	57.3721	57.4740	57.5949	+0.0194
问题四	h	换热强度	51.1001	51.0920	51.0855	-0.0014
	h_m	表面传质	51.3832	51.0920	50.8712	-0.0501
	D	内部扩散	56.0919	51.0920	47.0123	-0.8885
	β	平衡边界	50.9893	51.0920	51.2233	+0.0229

整重算。长期平台水分量为 $Y = 0.0499875$ ，结果见表13。

表 13 平衡边界闭合的宽情景

β	长期 C_{eq}	问题三 t_*/h	问题四 t_*/h
0.5	0.0250	57.0650	50.7478
1.0	0.0500	57.4740	51.0920
2.0	0.1000	61.2846	55.0980
2.8	0.1400	85.6727	$> 72^a$

^a72 h 时 $C_{max} = 0.15185$ ；半径观测在此处结束。

当长期 C_{eq} 由 0.05 提高到 0.10 时，两问终点分别延后 3.81 h 和 4.01 h；提高到 0.14 后，问题三终点增至 85.67 h，问题四在 72 h 半径观测范围内仍未达标。若长期 $C_{eq} \geq 0.15$ ，在持续平台边界下模型将趋向不低于阈值的平衡状态，严格停机事件不再出现。基准点附近较小的 S_β 只说明局部扰动影响有限，边界映射本身仍是长期预测的主要结构来源。表中数值是给定 β 下的条件结果，不表示其概率范围。

7.3 参数敏感性排序与机理解释

图9(a)(b) 展示正、负扰动引起的终点相对变化，图9(c) 进一步保留影响方向。两问均有

$$|S_D| > |S_{h_m}| > |S_\beta| > |S_h|. \quad (47)$$

扩散率尺度的 S_D 约为 -0.89 ，即在基准点附近，扩散率提高 1% 大致对应临界时间降低 0.89%；它是四个局部参数中影响最大的一个。这与临界阶段仍存在显著内部含水率梯度的结果一致。 h_m 位居第二，但问题四中的 $|S_{h_m}|$ 由问题三的 0.1142 降至 0.0501：收缩缩短内部路径并提高面积体积比后，终点对外膜传质的局部响应减弱。

S_h 在两问中均约为 -0.0014 ，原因是终点阶段温场已接近环境平台。 $S_\beta > 0$ ，增大 β 会提高等效平衡含水率、减小表面排湿驱动力。温度依赖影响早期扩散率演化，而终点对 h 本身的局部响应很弱，二者对应不同阶段。

正负扰动的响应还具有轻微不对称性。例如问题三中 D 降低 10% 使终点延后 9.8447%，而提高 10% 只使其提前 8.0165%；问题四对应 +9.7859% 和 -7.9850% 。这是变物性、时变边界与事件判停共同造成的非线性，局部灵敏度系数适用于基准点邻域。

图9(a)(b) 在扰动端点标出终点相对变化， D 的正负扰动存在约 1.8 个百分点的不对称；(c) 保留两问数值和统一排序；(d) 区分数值、输入/边界及参数/结构三类量级。 D 的排序针对基准附近的四个局部参数，边界闭合的宽情景则刻画模型口径改变后的终点变化。

7.4 情景稳健性与误差层级

参数扰动之外，本文还比较输入延拓、半径插值、端面条件和模型结构情景。图9(d) 以对数轴给出临界时间最大绝对差：纯数值离散约为 10^{-1} s 量级，问题四半径插值为 12.61 s，端面配对情景约为 7.49 s 和 0.028 s，环境延拓约为 10^3 s， $\pm 10\%$ 参数扰动约为 2×10^4 s，而机制或因素对照可达 10^5 s 量级。不同层级相差数个数量级，说明继续增加终点求根小数位不会弥补边界和结构假设的不确定性。

具体而言，4 h 后若保持末次观测而非采用 3–4 h 均值，问题三、四分别提前约 18.28 min 和 16.07 min；问题四将半径线性插值改为 PCHIP，终点仅提前 12.61 s。冻结扩散率中的温度因子会把问题三终点推迟至 128.9902 h，说明该经验模型中的温度依赖会影响完整过程。这一消融远离基准模型，用于解释机制，与 $\pm 10\%$ 局部参数扰动分开比较。

若用于实际预测，应优先通过实验校准 $D(C, T)$ 、平衡含水率关系和表面传质系数，并补充长期环境与端面边界信息。当前网格和时间容差对终点的影响已小于这些物理输入。

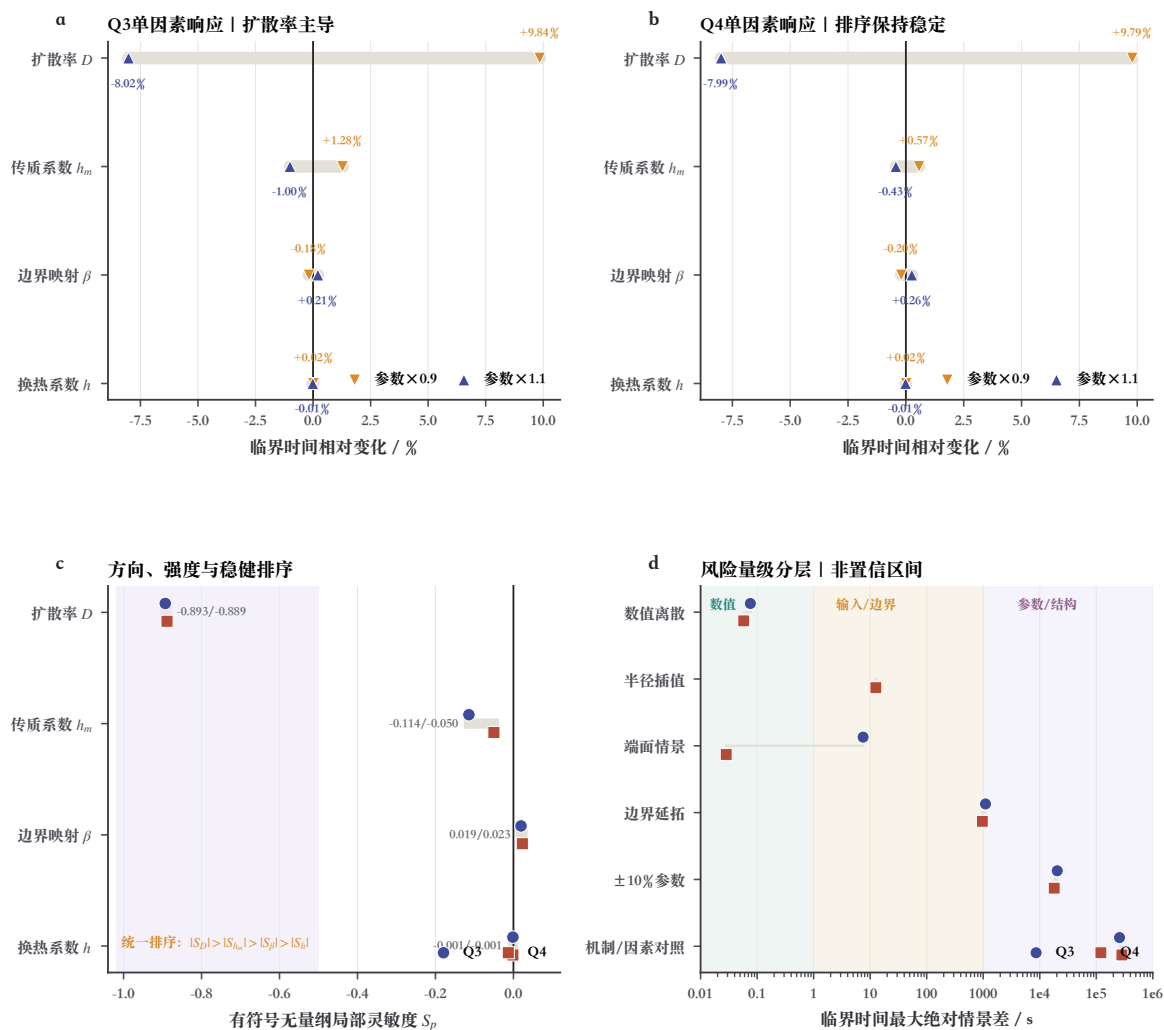


图9 终点灵敏度与风险层级

八、模型评价与推广

8.1 模型优点

- 1) **守恒口径统一。**从干固体和水分两条连续方程出发，区分干基含水率与单位体积浓度；问题四在材料坐标中不重复添加压缩项，并用累计收支验证。
- 2) **四问衔接清楚。**问题一、二、四从相同原始初态独立计算，问题三沿问题二轨迹继续；固定域、耦合、事件和收缩的新增因素可逐层解释。
- 3) **判停方式稳健。**终点由全域最大含水率决定，并以非中心湿峰反例排除“只看中心”的程序漏洞；临界等号与严格达标整数秒分开报告。
- 4) **验证来源相互独立。**解析基线、独立有限元、网格与时间收敛、累计守恒和输入留出分别检查方程、离散、实现与数据处理。

8.2 模型局限

- 1) 缺少药材内部温湿度和总质量实测，无法校准 $C_{eq} = G(Y, T)$ 、物性误差或给出统计置信区间。
- 2) 主热方程没有蒸发潜热、水分携带焓、辐射和形变功，属于有效显热模型，而非完整多相能量模型。
- 3) 一维主模型假定端面封闭且轴向均匀；二维对照只覆盖“端面与侧面同样暴露”这一情景，并非未知边界的严格上下界。
- 4) 问题四半径由观测直接指定，模型预测给定收缩轨迹下的温湿场，尚未建立含水率与收缩之间的闭合关系；经验密度作为有效热学参数使用。
- 5) 4 h 后的外部环境使用均值延拓，虽已作情景分析，但缺少后续观测验证。

8.3 改进与推广

若获得实验数据，可优先测量不同温湿环境下的吸附/解吸等温线与内部温湿剖面，用于辨识 C_{eq} 、 $D(C, T)$ 及其不确定性；进一步将潜热和水分焓通量纳入能量方程，并通过称重数据校准干质量与体积关系。若端面实际暴露，应使用经绝对误差验证的二维轴对称模型；若存在轴向各向异性或非均匀收缩，则扩展到二维各向异性或三维移动网格模型。

这一框架可用于根茎、果蔬等近圆柱多孔材料。更换物性、平衡含水率关系和几何观测后，守恒方程、材料坐标及全域事件的求解方式仍可沿用。用于工艺控制前，还需依据校准结果和测量误差设置停机裕度。

九、 结论

本文以干基含水率守恒为主线，完成了圆柱药材由预热、耦合干燥到径向收缩条件下的统一建模，主要结论如下：

- 1) 问题一的热、水分扩散长度在 30 min 时分别约为 1.743 cm 和 0.298 cm。数值结果与尺度判断一致：温度影响已进入内部，显著失水仍集中在表层，因此温度趋稳不能代表水分均匀。
- 2) 问题二中，3 h 时中心—表面温差仅 0.1169 °C，含水率差仍为 0.7581。冻结扩散率的温度因子会明显减少前三小时失水，说明所选经验模型中的温度依赖对早期过程影响显著。
- 3) 问题三以全域最大含水率定义事件，得到等号临界时间 57.4740 h，首个严格达标整数秒为 206907 s。平均值判停会提前约 21.45 h，说明最后湿点决定了实际终点。
- 4) 问题四在材料坐标中得到临界时间 51.0920 h。在统一外生半径轨迹下，四组条件对照表明，相对问题三缩短的 6.3820 h 同时包含几何、物性和交互影响。两问对扩散

率尺度的局部灵敏度均约为 -0.89 。

解析基线、网格收敛、独立有限元和累计守恒均支持上述数值结果。宽情景计算进一步表明，当长期平衡含水率接近 0.15 时，终点会显著延后，甚至超出半径观测时域。实际应用应优先校准 $D(C, T)$ 、平衡含水率和表面传质关系，再结合安全裕度确定停机时间。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、程序审查、数值验证和图表排版，详细使用情况见支撑材料。

参考文献

- [1] Paolo Caccavale, Maria Valeria De Bonis, and Gianpaolo Ruocco. Conjugate heat and mass transfer in drying: A modeling review. Journal of Food Engineering, 176:28–35, May 2016.
- [2] S. M. A. Rahman, M. R. Islam, and A. S. Mujumdar. A study of coupled heat and mass transfer in composite food products during convective drying. Drying Technology, 25(7-8):1359–1368, August 2007.
- [3] J. Aprajeeta, R. Gopirajah, and C. Anandharamakrishnan. Shrinkage and porosity effects on heat and mass transfer during potato drying. Journal of Food Engineering, 144:119–128, January 2015.
- [4] F. N. Fritsch and R. E. Carlson. Monotone piecewise cubic interpolation. SIAM Journal on Numerical Analysis, 17(2):238–246, April 1980.
- [5] Pauli Virtanen et al. Scipy 1.0: fundamental algorithms for scientific computing in python. Nature Methods, 17(3):261–272, February 2020.
- [6] P. Kjellgren and J. Hyvärinen. An arbitrary lagrangian-eulerian finite element method. Computational Mechanics, 21(1):81–90, February 1998.

附录

附录 A 输入插值留出检验

图10(a)–(c) 比较留出观测与两种插值重建，(d)–(f) 给出逐时刻残差、RMSE、MAE和最不利留出点，(g) 再用持续值 RMSE 归一化三种不同量纲的输入。线性插值相对

持续值基线分别降低环境温度、环境含湿量和半径 RMSE 约 35.4%、36.6% 和 91.8%；PCHIP 没有稳定优势。留出点来自原观测时段，故该检验不评价 4 h 后的环境延拓。

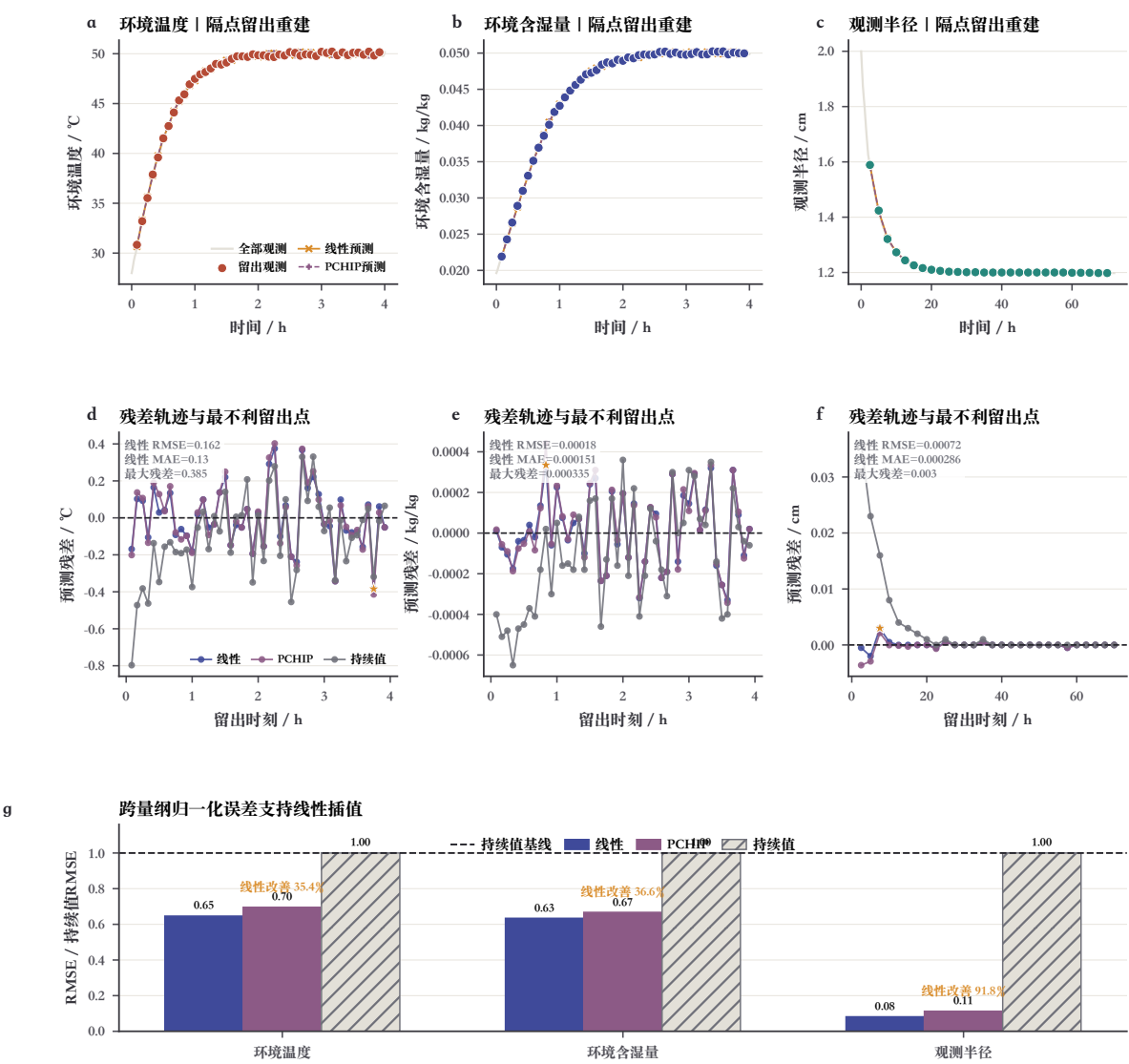


图 10 输入插值留出检验

附录 B 端面假设条件对照

二维轴对称对照设置两种情景：E0 端面绝热、不透湿；E1 端面与侧面采用相同 h 、 h_m 及环境条件。每个 E1 结果都与同径向网格、同容差、同最大步长的 E0 配对，避免把粗二维离散误差误算为端面效应。图11(a)(b) 给出 3 h 二维场差，(c) 把径向最大差投影为距端面距离的衰减曲线，(d)(e) 比较整根平均量与不同输出尺度，(f) 检查配对效应在细化情景下是否保持同一量级。因此，端面影响不能只由面积比例判断，也不能用单一中截面指标代表整根药材。

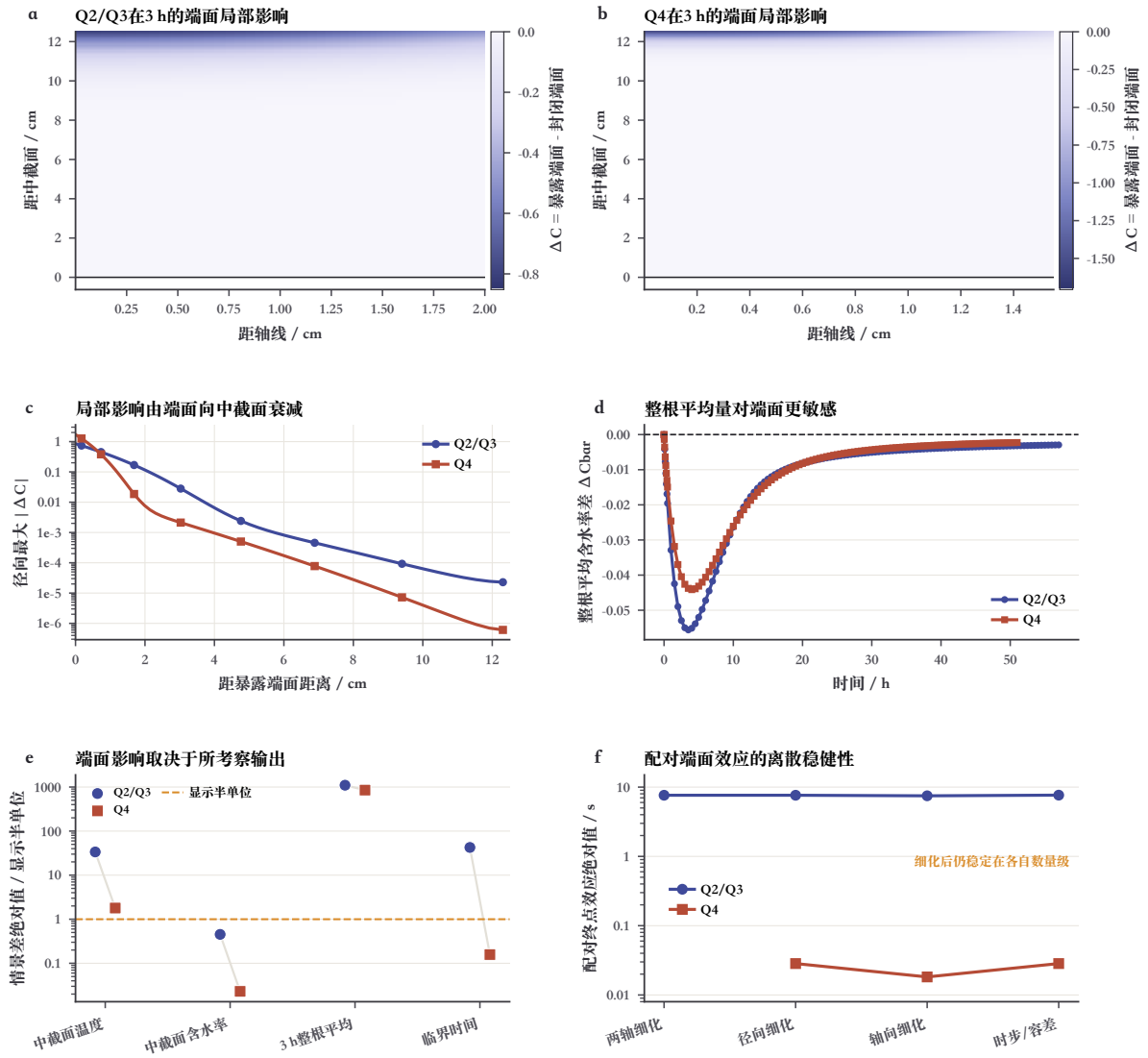


图 11 端面假设条件对照

端面暴露的影响主要集中在靠近两端的局部区域，中截面差异明显较小。配对细化后，问题三临界时间约提前 7.49 s，问题四约提前 0.0284 s；问题二前三小时的中截面温度最大差约 0.0017 °C。这说明端面假设对本文一维中截面结论影响有限，但整根平均量和端部局部场仍需在实际应用中单独校准。

附录 C 主要程序与结果文件

表 14 支撑文件说明

文件或目录	说明
scripts/question1.py-question4.py	四问主求解入口
src/	物性、边界、有限体积离散、重建与求解模块
config/model_config.json	几何、物性、边界、时间线及输入路径配置
validation/	解析基线、独立有限元、收敛、守恒、端面与反例检验
results/submission/result1.xlsx-result4.xlsx	按题目模板另存的最终结果文件，原始附件未覆盖
figures/	论文图的 PDF、SVG 及 600 dpi 位图版本
reports/final_result_validation.md	工作簿与主结果的最终验证记录
AI 工具使用详情.pdf	工具名称、使用过程及人工复核说明，提交前由参赛队确认

附录 D 核心求解源程序

下列程序包括问题一至问题四的主入口及其全部公共求解依赖。程序从项目根目录读取 config/model_config.json。生产结果所用参数为：

- 问题一： $N = 1280$ ，表面加密指数为 2， $rtol=1e-10$ ，最大步长 1 s；
- 问题二： $N = 1280$ ，表面加密指数为 2， $rtol=1e-10$ ，最大步长 2 s；
- 问题三：父轨迹为 q2_final， $rtol=1e-10$ ，最大步长 30 s；
- 问题四： $N = 1280$ ，表面加密指数为 2， $rtol=1e-10$ ，最大步长 30 s。

导出、绘图及全部验证程序随支撑材料一并提交，文件类别已列于表14。

4.1 四问主程序

文件：scripts/question1.py

Listing 1:

```

1  #!/usr/bin/env python3
2  """Run Q1 only; cache full precision and per-run metadata, not a final claim."""
3  import argparse
4  import sys
5  from pathlib import Path
6  sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
7
8  import numpy as np
9  from src.data_io import ROOT, load_config, verify_sources, save_json, source_fingerprint, digest
10 from src.boundary import Environment
11 from src.properties import Diffusivity
12 from src.fvm import RadialMesh, ScalarFVM
13 from src.solver import integrate_scalar
14 from src.reconstruction import reconstruct
15
16
17 def run_case(n=160, grading=1.0, rtol=1e-9, atol=1e-11, max_step=60, method="BDF", scheme="harmonic", label=None, store_cells=True,
18             perturbation=None):
19     cfg=load_config(); manifest=verify_sources(); env=Environment(cfg)
20     # Q1 always starts from the original uniform state, never another question's cache.
21     if cfg["problems"]["q1"] != {"properties": "q1", "radius": "fixed", "initialization": "fresh", "end_s": 1800}:
22         raise ValueError("Q1 timeline/property contract changed; review before running")
23     p=cfg["property_sets"]["q1"]; boundary=cfg["boundary"]; initial_cfg=cfg["initial"]
24     if (p["rho_kg_m3"]["slope_C"] != 0 or p["cp_J_kg_K"]["fraction_C_over_1_plus_C"] != 0
25         or p["k_W_m_K"]["fraction_C_over_1_plus_C"] != 0 or p["D_m2_s"]["thermal_exponent_K"] != 0):
26         raise ValueError("The Q1 scalar solver requires constant thermal properties and D(C)")
27     perturbation={} if perturbation is None else dict(perturbation)
28     if set(perturbation)-{"h_multiplier", "hm_multiplier", "D_multiplier", "beta_multiplier"}:
29         raise ValueError("Unknown Q1 sensitivity parameter")
30     if any(value<=0 for value in perturbation.values()):
31         raise ValueError("Sensitivity multipliers must be positive")
32     label=label or f"n{n}_g{grading:g}_f{method}_{scheme}_r{rtol:g}_h{max_step:g}"
33     folder=ROOT/"results/cache/q1"; folder.mkdir(parents=True, exist_ok=True)
34     npz_path=folder/(label+".npz")
35     meta_path=folder/(label+".json")
36     times=np.arange(0., 1801.)
37     radii=np.arange(21, dtype=float)*.001
38     mesh=RadialMesh(n, cfg["geometry"]["reference_radius_m"], grading)
39     meta={"label": label, "problem": "q1", "property_set": "q1", "start_s": 0, "end_s": 1800,
40         "n": n, "grading": grading, "internal_flux": scheme, "rtol": rtol, "atol": atol,
41         "max_step_s": max_step, "method": method, "source_fingerprint": source_fingerprint(),
42         "source_inputs": manifest["files"], "script_sha256": digest(__file__),
43         "perturbation": perturbation,
44         "status": "computed_pending_comparative_validation", "fields": {}}
45     cache={"time_s": times, "radius_m": radii, "cell_centres_m": mesh.centres, "faces_m": mesh.faces, "weights": mesh.weights}
46     specs=[("temperature", Diffusivity(p["k_W_m_K"]["offset"])),
47         boundary["heat_transfer_W_m2_K"]*perturbation.get("h_multiplier", 1.),
48         p["rho_kg_m3"]["offset"]*p["cp_J_kg_K"]["offset"], initial_cfg["temperature_C"],
49         ("moisture", Diffusivity(p["D_m2_s"]["prefactor"]*perturbation.get("D_multiplier", 1.), p["D_m2_s"] ["
50             moisture_exponent"])),
51         boundary["normalized_mass_transfer_m_s"]*perturbation.get("hm_multiplier", 1.), 1., initial_cfg["
52             moisture_dry_basis"])]
53     for field, law, transfer, capacity, initial in specs:
54         ambient_scale=perturbation.get("beta_multiplier", 1.) if field=="moisture" else 1.
55         model=ScalarFVM(mesh, law, transfer, lambda t, f=field, b=ambient_scale*b*env.value(t, f), capacity, scheme)
56         result, stats=integrate_scalar(model, initial, 1800, times, env.times, method, rtol, atol, max_step)
57         sampled=reconstruct(model, times, result[:, :n], radii, uniform_initial=initial)
58         cache[field]=sampled
59         cache[field+"_mean"]=np.sum(result[:, :n]*mesh.weights, axis=1)
60         cache[field+"_outward_integral"]=result[:, -1]
61     if store_cells:
62         cache[field+"_cells"]=result[:, :n]
63     meta["fields"][field]=stats

```

```

61         print(label,field,"done",round(stats["elapsed_s"],3),"s; balance",stats["independent_balance_max_abs"],flush=True)
62     np.savez_compressed(npz_path,**cache)
63     meta["cache_sha256"]=digest(npz_path)
64     save_json(meta_path,meta)
65     return label,cache,meta
66
67
68 if __name__=="__main__":
69     parser=argparse.ArgumentParser()
70     parser.add_argument("--n",type=int,default=160)
71     parser.add_argument("--grading",type=float,default=1.)
72     parser.add_argument("--rtol",type=float,default=1e-9)
73     parser.add_argument("--atol",type=float,default=1e-11)
74     parser.add_argument("--max-step",type=float,default=60.)
75     parser.add_argument("--method",choices=["BDF","Radau"],default="BDF")
76     parser.add_argument("--scheme",choices=["harmonic","kirchhoff"],default="harmonic")
77     parser.add_argument("--label")
78     args=parser.parse_args()
79     run_case(args.n,args.grading,args.rtol,args.atol,args.max_step,args.method,args.scheme,args.label)

```

文件: scripts/question2.py

Listing 2:

```

1  #!/usr/bin/env python3
2  """Q2: fresh appendix-three coupled trajectory; 3h milestone, not Q3 event."""
3  import argparse
4  import json
5  import sys
6  from pathlib import Path
7
8  sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
9
10 import numpy as np
11
12 from src.boundary import Environment
13 from src.coupled.model import CoupledFVM
14 from src.coupled.integrate import integrate
15 from src.data_io import ROOT, digest, load_config, save_json, source_fingerprint, verify_sources
16 from src.fvm import RadialMesh
17
18
19 def fingerprint():
20     result = source_fingerprint()
21     paths = sorted((ROOT/"src/coupled").glob("*.py"))+[Path(__file__)]
22     result.update({str(p.relative_to(ROOT)): digest(p) for p in paths})
23     return result
24
25
26 def read_case(label):
27     folder = ROOT/"results/cache/q23"
28     path = folder/(label+".npz")
29     metadata = json.loads((folder/(label+".json")).read_text())
30     if metadata["source_fingerprint"] != fingerprint() or metadata["cache_sha256"] != digest(path):
31         raise ValueError("Stale or altered Q2 cache: "+label)
32     with np.load(path) as raw:
33         cache = {key: raw[key] for key in raw.files}
34     return cache, metadata
35
36
37 def run_case(n=320, grading=2., rtol=1e-9, atol_temperature=1e-10, atol_moisture=1e-12,
38             max_step=10., method="BDF", label=None, perturbation=None):
39     config = load_config()

```

```

40     manifest = verify_sources()
41     expected = {"properties": "q23", "radius": "fixed", "initialization": "fresh",
42                "end": "q3_event", "paper_end_s": 10800}
43     if config["problems"]["q2"] != expected:
44         raise ValueError("Q2 timeline/property contract changed")
45     env = Environment(config)
46     mesh = RadialMesh(n, config["geometry"]["reference_radius_m"], grading)
47     model = CoupledFVM(mesh, config, env, "q23", perturbation)
48     cache, stats = integrate(model, end=10800., rtol=rtol, atol_temperature=atol_temperature,
49                             atol_moisture=atol_moisture, max_step=max_step, method=method)
50     label = label or f"q2_n{n}_g{grading:g}_{method:r}{rtol:g}_h{max_step:g}"
51     folder = ROOT/"results/cache/q23"
52     folder.mkdir(parents=True, exist_ok=True)
53     path = folder/(label+".npz")
54     np.savez_compressed(path, **cache)
55     metadata = {"label": label, "problem": "q2", "property_set": "q23", "initialization": "fresh",
56                "time_origin_s": 0., "start_s": 0., "end_s": 10800.,
57                "scope": "first_three_hours_only; full_workbook_end_pending_Q3_event",
58                "status": "computed_pending_comparative_validation", "n": n, "grading": grading,
59                "rtol": rtol, "atol_temperature": atol_temperature, "atol_moisture": atol_moisture,
60                "max_step_s": max_step, "method": method, "perturbation": dict(perturbation or {}),
61                "boundary_half_cell": "integral_C_at_local_mean_T_and_k_at_local_mean_C",
62                "source_fingerprint": fingerprint(), "source_inputs": manifest["files"],
63                "cache_sha256": digest(path), "statistics": stats}
64     save_json(folder/(label+".json"), metadata)
65     print(json.dumps({"label": label, "statistics": stats,
66                      "temperature_3h_centre_surface": cache["temperature"][-1, [0, -1]].tolist(),
67                      "moisture_3h_centre_surface": cache["moisture"][-1, [0, -1]].tolist()}, indent=2), flush=True)
68     return cache, metadata
69
70
71 if __name__ == "__main__":
72     parser = argparse.ArgumentParser()
73     parser.add_argument("--n", type=int, default=320)
74     parser.add_argument("--grading", type=float, default=2.)
75     parser.add_argument("--rtol", type=float, default=1e-9)
76     parser.add_argument("--max-step", type=float, default=10.)
77     parser.add_argument("--method", choices=["BDF", "Radau"], default="BDF")
78     parser.add_argument("--label")
79     args = parser.parse_args()
80     run_case(n=args.n, grading=args.grading, rtol=args.rtol, max_step=args.max_step,
81             method=args.method, label=args.label)

```

文件: scripts/question3.py

Listing 3:

```

1  #!/usr/bin/env python3
2  """Q3 event from a provenance-checked Q2 full-precision checkpoint."""
3  import argparse
4  import json
5  import sys
6  from pathlib import Path
7
8  sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
9  import numpy as np
10
11 from scripts.question2 import read_case as read_q2, fingerprint as q2_fingerprint
12 from src.coupled.model import CoupledFVM
13 from src.data_io import ROOT, digest, load_config, save_json, verify_sources
14 from src.events.integration import ScenarioEnvironment, continue_to_event
15 from src.fvm import RadialMesh
16

```

```

17
18 def fingerprint():
19     fp = q2_fingerprint()
20     fp.update({str(p.relative_to(ROOT)): digest(p) for p in
21               sorted((ROOT/"src/events").glob("*.py"))+[Path(__file__)]})
22     return fp
23
24
25 def read_case(label):
26     folder = ROOT/"results/cache/q3"
27     path = folder/(label+".npz")
28     meta = json.loads((folder/(label+".json")).read_text())
29     if meta["source_fingerprint"] != fingerprint() or meta["cache_sha256"] != digest(path):
30         raise ValueError("Stale Q3 case: "+label)
31     _, parent = read_q2(meta["q2_parent_label"])
32     if meta["q2_parent_sha256"] != parent["cache_sha256"]:
33         raise ValueError("Q2 continuation checkpoint changed")
34     with np.load(path) as raw:
35         return {k: raw[k] for k in raw.files}, meta
36
37
38 def run_case(parent="q2_final", label="q3_trial", max_step=60., rtol=1e-10,
39             method="BDF", output_step=60., scenario="main", limit=604800.):
40     config = load_config()
41     manifest = verify_sources()
42     if config["problems"]["q3"] != {"properties": "q23", "radius": "fixed",
43                                     "initialization": "q2_full_precision_same_config_or_fresh", "time_origin_s": 0,
44                                     "end": "global_moisture_event"}:
45         raise ValueError("Q3 timeline contract changed")
46     cache, meta = read_q2(parent)
47     if (meta["property_set"], meta["initialization"], meta["end_s"], meta["time_origin_s"]) != ("q23", "fresh", 10800., 0.)
48         :
49         raise ValueError("Wrong Q2 checkpoint timeline/properties")
50     assert cache["time_s"][-1] == 10800 and np.array_equal(cache["profile_state"][-1], cache["final_state"])
51     mesh = RadialMesh(meta["n"], config["geometry"]["reference_radius_m"], meta["grading"])
52     assert np.array_equal(mesh.centres, cache["cell_centres_m"])
53     model = CoupledFVM(mesh, config, ScenarioEnvironment(config, scenario), "q23", meta["perturbation"])
54     result, stats = continue_to_event(model, cache["final_state"], max_step=max_step,
55                                     rtol=rtol, method=method, output_step=output_step, limit=limit)
56     folder = ROOT/"results/cache/q3"
57     folder.mkdir(parents=True, exist_ok=True)
58     path = folder/(label+".npz")
59     np.savez_compressed(path, **result)
60     metadata = {"label": label, "property_set": "q23", "physical_time_origin_s": 0,
61                "continuation_start_s": 10800., "q2_parent_label": parent,
62                "q2_parent_sha256": meta["cache_sha256"], "n": meta["n"], "grading": meta["grading"],
63                "perturbation": meta["perturbation"], "rtol": rtol, "atol_temperature": 1e-11,
64                "atol_moisture": 1e-13, "max_step_s": max_step, "method": method,
65                "output_step_s": output_step, "boundary_scenario": scenario,
66                "source_fingerprint": fingerprint(), "cache_sha256": digest(path),
67                "source_inputs": manifest["files"], "statistics": stats,
68                "status": "computed_pending_Q3_convergence_and_independent_verification"}
69     save_json(folder/(label+".json"), metadata)
70     print(json.dumps({"label": label, "statistics": stats}), flush=True)
71     return result, metadata
72
73 if __name__ == "__main__":
74     parser = argparse.ArgumentParser()
75     parser.add_argument("--parent", default="q2_final")
76     parser.add_argument("--label", default="q3_trial")
77     parser.add_argument("--max-step", type=float, default=60.)
78     parser.add_argument("--rtol", type=float, default=1e-10)
79     parser.add_argument("--method", choices=["BDF", "Radau"], default="BDF")

```

```

80     parser.add_argument("--output-step", type=float, default=60.)
81     parser.add_argument("--scenario", choices=["main", "last_observation", "mean_9000_14400"], default="main")
82     parser.add_argument("--limit", type=float, default=604800.)
83     run_case(**vars(parser.parse_args()))

```

文件: scripts/question4.py

Listing 4:

```

1  #!/usr/bin/env python3
2  """Q4 and controlled radius/property combinations, always from original t=0."""
3  import argparse
4  import json
5  import sys
6  from pathlib import Path
7  sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
8  import numpy as np
9
10 from scripts.question3 import fingerprint as previous_fingerprint
11 from src.data_io import ROOT, digest, load_config, save_json, verify_sources
12 from src.events.integration import ScenarioEnvironment
13 from src.fvm import RadialMesh
14 from src.moving.model import MovingCoupled, Radius
15 from src.moving.integration import integrate
16
17
18 def fingerprint():
19     fp=previous_fingerprint()
20     fp.update({str(p.relative_to(ROOT)):digest(p) for p in sorted((ROOT/"src/moving").glob("*.py"))+[Path(__file__)]})
21     return fp
22
23
24 def read_case(label):
25     folder=ROOT/"results/cache/q4"; path=folder/(label+".npz")
26     m=json.loads((folder/(label+".json")).read_text())
27     if m["source_fingerprint"]!=fingerprint() or m["cache_sha256"]!=digest(path):
28         raise ValueError("Stale Q4 cache: "+label)
29     with np.load(path) as raw: return {k:raw[k] for k in raw.files},m
30
31
32 def run_case(label="q4_trial",n=320,group="q4",geometry="linear",extension="error",
33             scenario="main",max_step=30.,rtol=1e-10,method="BDF",limit=259200.,perturbation=None):
34     cfg=load_config(); manifest=verify_sources()
35     expected={"properties":"q4","radius":"observed_shrinkage","initialization":"fresh","end":"
36         global_moisture_event_or_radius_data_end"}
37     if cfg["problems"]["q4"]!=expected: raise ValueError("Q4 timeline contract changed")
38     radius=Radius(cfg,geometry,extension)
39     model=MovingCoupled(RadialMesh(n,.02,2.),cfg,ScenarioEnvironment(cfg,scenario),radius,group,perturbation)
40     c,stats=integrate(model,limit=limit,max_step=max_step,rtol=rtol,method=method)
41     assert np.array_equal(c["profile_state"][0],np.tile([28.,2.55],n))
42     folder=ROOT/"results/cache/q4"; folder.mkdir(parents=True,exist_ok=True)
43     path=folder/(label+".npz"); np.savez_compressed(path,**c)
44     m={"label":label,"n":n,"grading":2.,"property_set":group,"geometry":geometry,
45       "radius_extension":extension,"boundary_scenario":scenario,"initialization":"fresh",
46       "physical_time_origin_s":0.,,"rtol":rtol,"atol_temperature":1e-11,"atol_moisture":1e-13,
47       "max_step_s":max_step,"method":method,"requested_limit_s":limit,"output_step_s":60.,
48       "perturbation":dict(perturbation or {}),"source_fingerprint":fingerprint(),
49       "cache_sha256":digest(path),"source_inputs":manifest["files"],"statistics":stats,
50       "status":"computed_pending_Q4_numerical_and_independent_validation"}
51     save_json(folder/(label+".json"),m)
52     print(json.dumps({"label":label,"statistics":stats}),flush=True)
53     return c,m

```

```

54
55 if __name__=="__main__":
56     p=argparse.ArgumentParser()
57     p.add_argument("--label",default="q4_trial");p.add_argument("--n",type=int,default=320)
58     p.add_argument("--group",choices=["q4","q23"],default="q4")
59     p.add_argument("--geometry",choices=["linear","pchip","fixed"],default="linear")
60     p.add_argument("--extension",choices=["error","hold_last"],default="error")
61     p.add_argument("--scenario",choices=["main","last_observation","mean_9000_14400"],default="main")
62     p.add_argument("--max-step",type=float,default=30.);p.add_argument("--rtol",type=float,default=1e-10)
63     p.add_argument("--method",choices=["BDF","Radau"],default="BDF")
64     p.add_argument("--limit",type=float,default=259200.)
65     run_case(**vars(p.parse_args()))

```

4.2 公共求解模块

文件: src/__init__.py

Listing 5:

```

1 """Radial effective heat and dry-basis moisture transport components."""

```

文件: src/data_io.py

Listing 6:

```

1 """Configuration, source integrity and generated-artifact helpers."""
2 import hashlib
3 import json
4 from pathlib import Path
5
6 ROOT = Path(__file__).resolve().parents[1]
7
8
9 def digest(path):
10     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
11
12
13 def load_config():
14     return json.loads((ROOT / "config/model_config.json").read_text(encoding="utf-8"))
15
16
17 def verify_sources():
18     manifest = json.loads((ROOT / "reports/input_manifest.json").read_text(encoding="utf-8"))
19     for item in manifest["files"]:
20         if digest(item["path"]) != item["sha256"]:
21             raise ValueError("Input differs from stage0 manifest: " + item["path"])
22     return manifest
23
24
25 def save_json(path, value):
26     path = Path(path)
27     path.parent.mkdir(parents=True, exist_ok=True)
28     path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + "\n", encoding="utf-8")
29
30
31 def source_fingerprint():
32     paths = sorted((ROOT / "src").glob("*.py"))
33     paths += [ROOT / "config/model_config.json"]
34     return {str(p.relative_to(ROOT)): digest(p) for p in paths}

```

文件: src/boundary.py

Listing 7:

```
1  """Measured environment and true surface values from half-cell + Robin closure."""
2  from pathlib import Path
3
4  import numpy as np
5  from openpyxl import load_workbook
6  from scipy.optimize import brentq
7
8
9  class Environment:
10     def __init__(self, config):
11         path = Path(config["source_directory"]) / "附件1.xlsx"
12         wb = load_workbook(path, read_only=True, data_only=True)
13         self.data = np.array(list(wb.active.values)[1:], dtype=float)
14         wb.close()
15         self.times = self.data[:, 0]
16         boundary = config["boundary"]
17         a, b = boundary["mean_window_s"]
18         self.platform = self.data[(self.times >= a) & (self.times <= b), 1:].mean(axis=0)
19         self.beta = boundary["beta"]
20
21     def value(self, t, field):
22         t = np.asarray(t)
23         if np.any(t < 0):
24             raise ValueError("Environment time precedes initial state")
25         col = 1 if field == "temperature" else 2
26         value = np.interp(t, self.times, self.data[:, col])
27         value = np.where(t > self.times[-1], self.platform[col-1], value)
28         return value if col == 1 else value*self.beta
29
30
31     def surface_value(inner, ambient, half_width, coefficient, transfer):
32         """Integral of D across half-cell equals Robin flux times half-width.
33
34         This integral-resistance closure is also used with harmonic interior fluxes.
35         It differs from assuming the last cell centre is the physical surface.
36         """
37         inner, ambient = np.broadcast_arrays(np.asarray(inner, float), np.asarray(ambient, float))
38         if transfer == 0:
39             return inner.copy()
40         if coefficient.a == 0:
41             z = transfer*half_width/coefficient.prefactor
42             return (inner + z*ambient)/(1+z)
43         if np.any(inner <= 0) or np.any(ambient <= 0):
44             raise ValueError("Moisture surface bracket must remain positive")
45         z = transfer*half_width/coefficient.prefactor
46         lo, hi = np.minimum(inner, ambient).copy(), np.maximum(inner, ambient).copy()
47         initial_D = coefficient.value(inner)/coefficient.prefactor
48         surface = (initial_D*inner+z*ambient)/(initial_D+z)
49         primitive_inner = coefficient.primitive_scaled(inner)
50         for _ in range(10):
51             residual = primitive_inner-coefficient.primitive_scaled(surface)-z*(surface-ambient)
52             deriv = -coefficient.value(surface)/coefficient.prefactor-z
53             lo = np.where(residual > 0, surface, lo)
54             hi = np.where(residual < 0, surface, hi)
55             proposal = surface-residual/deriv
56             # Allow exact root/endpoints; replace only genuinely out-of-bracket iterates.
57             proposal = np.where((proposal < lo) | (proposal > hi), .5*(lo+hi), proposal)
58             if np.max(np.abs(proposal-surface)) < 2e-13:
59                 surface = proposal
60             break
```

```

61     surface = proposal
62     residual = primitive_inner-coefficient.primitive_scaled(surface)-z*(surface-ambient)
63     if np.any(np.abs(residual) > 3e-12):
64         flat = surface.reshape(-1)
65         for i in np.flatnonzero(np.abs(residual.reshape(-1)) > 3e-12):
66             ci, ce = inner.flat[i], ambient.flat[i]
67             flat[i] = brentq(lambda s: coefficient.primitive_scaled(ci)-coefficient.primitive_scaled(s)-z*(s-ce), min(ci,ce
68             ), max(ci,ce), xtol=5e-14)
69             surface = flat.reshape(inner.shape)
69     return surface

```

文件: src/properties.py

Listing 8:

```

1  """Appendix parameters; thermal effective density is not dry-solid density."""
2  import numpy as np
3  from scipy.special import expi
4
5
6  class PropertyLaw:
7      def __init__(self, config, group):
8          self.group = group
9          self.p = config["property_sets"][group]
10
11      def evaluate(self, C, T_C):
12          C, T_C = np.asarray(C), np.asarray(T_C)
13          if np.any(C <= 0) or np.any(T_C + 273.15 <= 0):
14              raise ValueError("Nonphysical accepted/trial property state: C<=0 or T_K<=0")
15          p = self.p
16          rho = p["rho_kg_m3"]["offset"] + p["rho_kg_m3"]["slope_C"] * C
17          cp = p["cp_J_kg_K"]["offset"] + p["cp_J_kg_K"]["fraction_C_over_1_plus_C"] * C/(1+C)
18          k = p["k_W_m_K"]["offset"] + p["k_W_m_K"]["fraction_C_over_1_plus_C"] * C/(1+C)
19          d = p["D_m2_s"]
20          D = d["prefactor"] * np.exp(-d["moisture_exponent"]/C-d["thermal_exponent_K"]/(T_C+273.15))
21          return rho*cp, k, D
22
23
24  class Diffusivity:
25      """Scalar field coefficient; Q1 D(C), or a constant validation coefficient."""
26      def __init__(self, prefactor, a=0.0):
27          self.prefactor, self.a = float(prefactor), float(a)
28
29      def value(self, u):
30          u = np.asarray(u)
31          if self.a and np.any(u <= 0):
32              raise ValueError("Nonpositive moisture state, no silent clipping")
33          return np.ones_like(u, dtype=float)*self.prefactor if not self.a else self.prefactor*np.exp(-self.a/u)
34
35      def derivative(self, u):
36          return np.zeros_like(u, dtype=float) if not self.a else self.value(u)*self.a/np.asarray(u)**2
37
38      def primitive_scaled(self, u):
39          u = np.asarray(u)
40          if not self.a:
41              return u
42          return u*np.exp(-self.a/u)+self.a*expi(-self.a/u)
43
44      def integral(self, lo, hi):
45          return self.prefactor*(self.primitive_scaled(hi)-self.primitive_scaled(lo))

```

文件: src/fvm.py

Listing 9:

```

1  """Conservative annular scalar transport with analytic tridiagonal Jacobian.
2
3      capacity * u_t = (r*a(u)*u_r)_r/r.
4  Only the scalar/decoupled Q1 solver is validated in this milestone.
5  """
6  from dataclasses import dataclass
7
8  import numpy as np
9  from scipy.sparse import diags
10
11  from .boundary import surface_value
12
13
14  @dataclass
15  class RadialMesh:
16      n: int
17      radius: float
18      grading: float = 1.0
19
20      def __post_init__(self):
21          if self.n < 3 or self.radius <= 0 or self.grading < 1:
22              raise ValueError("Invalid radial mesh")
23          uniform = np.linspace(0, 1, self.n+1)
24          self.faces = self.radius*(1-(1-uniform)**self.grading)
25          self.centres = .5*(self.faces[1:]+self.faces[:-1])
26          self.volumes = .5*np.diff(self.faces**2) # per unit 2*pi*L
27          self.weights = 2*self.volumes/self.radius**2
28          self.left_distances = self.faces[1:-1]-self.centres[:-1]
29          self.right_distances = self.centres[1:]-self.faces[1:-1]
30          self.separations = np.diff(self.centres)
31          self.half_width = self.faces[-1]-self.centres[-1]
32
33
34  class ScalarFVM:
35      def __init__(self, mesh, coefficient, transfer, ambient, capacity=1.0, flux_scheme="harmonic"):
36          self.mesh, self.coefficient = mesh, coefficient
37          self.transfer, self.ambient, self.capacity = transfer, ambient, capacity
38          self.flux_scheme = flux_scheme
39          if flux_scheme not in ("harmonic", "kirchhoff"):
40              raise ValueError("Unknown face flux")
41          if transfer < 0 or capacity <= 0 or coefficient.prefactor < 0:
42              raise ValueError("Invalid transport coefficient/capacity")
43
44      def boundary(self, t, inner):
45          return surface_value(inner, self.ambient(t), self.mesh.half_width, self.coefficient, self.transfer)
46
47      def surface_flux(self, t, inner):
48          return self.transfer*(self.boundary(t, inner)-self.ambient(t))
49
50      def face_flux_and_derivatives(self, t, u):
51          m, law = self.mesh, self.coefficient
52          if law.prefactor == 0:
53              if self.transfer != 0:
54                  raise ValueError("Zero diffusion test requires insulated surface")
55              zeros = np.zeros(m.n-1)
56              return zeros, zeros.copy(), zeros.copy(), 0.0, 0.0
57          D = law.value(u)
58          if self.flux_scheme == "kirchhoff":
59              flux = -law.integral(u[:-1], u[1:])/m.separations
60              left, right = D[:-1]/m.separations, -D[1:]/m.separations
61          else:
62              resistance = m.left_distances/D[:-1]+m.right_distances/D[1:]
63              flux = (u[:-1]-u[1:])/resistance

```

```

64         Dprime = law.derivative(u)
65         left = 1/resistance + flux/resistance*m.left_distances*Dprime[:-1]/D[:-1]**2
66         right = -1/resistance + flux/resistance*m.right_distances*Dprime[1:]/D[1:]**2
67         surface = self.boundary(t,u[-1])
68         q_s = self.transfer*(surface-self.ambient(t))
69         derivative_s = self.transfer*D[-1]/(law.value(surface)+self.transfer*m.half_width)
70         return flux, left, right, q_s, derivative_s
71
72     def rhs(self, t, state):
73         n, m = self.mesh.n, self.mesh
74         flux, _, _, qs, _ = self.face_flux_and_derivatives(t, state[:n])
75         face_total = np.concatenate(([0.0], m.faces[1:-1]*flux, [m.radius*qs]))
76         du = -np.diff(face_total)/(m.volumes*self.capacity)
77         cumulative_outward = 2*qs/(m.radius*self.capacity)
78         return np.r_[du, cumulative_outward]
79
80     def jacobian(self, t, state):
81         m, n = self.mesh, self.mesh.n
82         _, left, right, _, ds = self.face_flux_and_derivatives(t, state[:n])
83         area = m.faces[1:-1]
84         scale = m.volumes*self.capacity
85         diagonal = np.zeros(n+1)
86         diagonal[:-2] -= area*left/scale[:-1]
87         diagonal[1:-1] += area*right/scale[1:]
88         diagonal[n-1] -= m.radius*ds/scale[-1]
89         lower = np.r_[area*left/scale[1:], 2*ds/(m.radius*self.capacity)]
90         upper = np.r_[area*right/scale[-1], 0.0]
91         return diags([lower, diagonal, upper], [-1,0,1], format="csc")

```

文件: src/solver.py

Listing 10:

```

1  """Piecewise adaptive stiff integration and independent flux quadrature."""
2  import time
3  import numpy as np
4  from scipy.integrate import solve_ivp, BDF
5  from numpy.polynomial.legendre import leggauss
6
7
8  class InitializedBDF(BDF):
9      """Initialize unused work rows; leave SciPy's BDF formulas unchanged.
10
11      SciPy 1.13.1 uses np.empty for D and initially assigns only D[0:2].
12      Its first difference update reads the not-yet-used D[2], potentially
13      emitting an invalid-subtract warning from arbitrary allocator contents.
14      These higher differences are filled before they affect the solution.
15      Explicit initialization makes warning-as-error runs deterministic.
16      No warnings or nonphysical accepted states are suppressed.
17      """
18      def __init__(self, *args, **kwargs):
19          super().__init__(*args, **kwargs)
20          self.D[2:] = 0.0
21
22
23  def integrate_scalar(model, initial, end, sample_times, knots, method="BDF", rtol=1e-9, atol=1e-11, max_step=60.,
24                      balance_order=8):
25      started=time.perf_counter()
26      n=model.mesh.n
27      initial=np.broadcast_to(np.asarray(initial,float),(n,)).copy()
28      state=np.r_[initial,0.0]
29      sample_times=np.asarray(sample_times,float)
30      if np.any(np.diff(sample_times)<=0) or sample_times[0]<0 or sample_times[-1]>end:

```

```

30         raise ValueError("Invalid evaluation timeline")
31     cuts=np.unique(np.r_[0.,np.asarray(knots)[(np.asarray(knots)>0)&(np.asarray(knots)<end)],end])
32     output=np.empty((len(sample_times),n+1))
33     gx,gw=leggauss(balance_order)
34     accepted=[]
35     quadrature_out=0.0
36     max_quadrature_balance=0.0
37     nfev=njev=nlu=0
38     for a,b in zip(cuts[:-1],cuts[1:]):
39         integrator=InitializedBDF if method=="BDF" else method
40         result=solve_ivp(model.rhs,(a,b),state,method=integrator,rtol=rtol,atol=np.r_[np.full(n,atol),min(atol,1e-11)],jac=
model.jacobian,dense_output=True,max_step=max_step)
41         if not result.success:
42             raise RuntimeError(result.message)
43         if not np.isfinite(result.y).all():
44             raise RuntimeError("Nonfinite accepted solution")
45         state=result.y[:,-1]
46         indices=np.flatnonzero((sample_times>a)&(sample_times<=b))
47         if len(indices):
48             output[indices]=result.sol(sample_times[indices]).T
49         steps=np.diff(result.t)
50         accepted.extend(steps.tolist())
51         nfev+=result.nfev; njev+=result.njev; nlu+=result.nlu
52         mid=.5*(result.t[1:]+result.t[:-1]); half=.5*steps
53         tq=(mid[:,None]+half[:,None]*gx).reshape(-1)
54         inner=result.sol(tq)[n-1]
55         rate=2*model.surface_flux(tq,inner)/(model.mesh.radius*model.capacity)
56         increments=half*np.sum(rate.reshape(-1,balance_order)*gw,axis=1)
57         cumulative=quadrature_out+np.cumsum(increments)
58         weighted_means=np.sum(model.mesh.weights[:,None]*result.y[:,n,1:],axis=0)
59         residual=weighted_means-np.sum(model.mesh.weights*initial)+cumulative
60         max_quadrature_balance=max(max_quadrature_balance,float(np.max(np.abs(residual))))
61         quadrature_out=float(cumulative[-1])
62     mean=np.sum(output[:,n]*model.mesh.weights,axis=1)
63     algebraic_balance=mean-np.sum(model.mesh.weights*initial)+output[:,-1]
64     stats={"method":method,"rtol":rtol,"atol":atol,"max_step_cap_s":max_step,
65           "accepted_steps":len(accepted),"accepted_max_step_s":max(accepted),
66           "accepted_min_step_s":min(accepted),"nfev":nfev,"njev":njev,"nlu":nlu,
67           "balance_quadrature_order":balance_order,"independent_balance_max_abs":max_quadrature_balance,
68           "augmented_balance_max_abs":float(np.max(np.abs(algebraic_balance))),
69           "elapsed_s":time.perf_counter()-started}
70     return output,stats

```

文件: src/reconstruction.py

Listing 11:

```

1  """Axis/surface recovery and physical-radius sampling of cell-centred states."""
2  import numpy as np
3  from scipy.interpolate import PchipInterpolator
4
5
6  def reconstruct(model, times, cell_values, radii, uniform_initial=None):
7      m = model.mesh
8      radii = np.asarray(radii)
9      if np.any(radii < 0) or np.any(radii > m.radius):
10         raise ValueError("Sample outside physical material")
11     r0, r1 = m.centres[:2]
12     axis = (r1*r1*cell_values[:,0]-r0*r0*cell_values[:,1])/(r1*r1-r0*r0)
13     surface = model.boundary(times,cell_values[:,-1])
14     points = np.r_[0.,m.centres,m.radius]
15     values = np.column_stack((axis,cell_values,surface))
16     sampled = PchipInterpolator(points,values,axis=1,extrapolate=False)(radii)

```

```

17     near_axis = radii < r0
18     # Even quadratic on the central interval enforces derivative zero at the axis.
19     if np.any(near_axis):
20         curvature = (cell_values[:,1]-cell_values[:,0])/(r1*r1-r0*r0)
21         sampled[:,near_axis] = axis[:,None]+curvature[:,None]*radii[near_axis]**2
22     if uniform_initial is not None:
23         sampled[np.asarray(times)==0]=uniform_initial
24     return sampled

```

文件: src/coupled/__init__.py

Listing 12:

```

1 """Q23 coupled transport, versioned separately from the accepted Q1 core."""

```

文件: src/coupled/model.py

Listing 13:

```

1 """Fixed-domain, coupled annular FVM with an analytic block Jacobian.
2
3 Surface moisture resistance integrates D(C,T_face) in C, holding the local
4 half-cell mean temperature fixed, not claiming a global Kirchhoff transform.
5 Heat resistance uses k at the half-cell mean moisture. Both surface states
6 are solved together. Every coefficient is updated from the current state.
7 """
8 import numpy as np
9 from scipy.optimize import brentq
10 from scipy.sparse import coo_matrix
11
12 from src.properties import PropertyLaw, Diffusivity
13
14
15 class CoupledFVM:
16     def __init__(self, mesh, config, environment, group="q23", perturbation=None):
17         self.mesh, self.config, self.environment = mesh, config, environment
18         self.group = group
19         self.law = PropertyLaw(config, group)
20         self.p = self.law.p
21         self.options = dict(perturbation or {})
22         allowed = {"h_multiplier", "hm_multiplier", "D_multiplier", "beta_multiplier",
23                   "freeze_D_temperature", "freeze_thermal_moisture"}
24         if set(self.options) - allowed:
25             raise ValueError("Unknown perturbation")
26         for key, value in self.options.items():
27             if key.endswith("multiplier") and value <= 0:
28                 raise ValueError("Nonpositive multiplier")
29         self.h = config["boundary"]["heat_transfer_W_m2_K"] * self.options.get("h_multiplier", 1.)
30         self.hm = config["boundary"]["normalized_mass_transfer_m_s"] * self.options.get("hm_multiplier", 1.)
31         self.beta = self.options.get("beta_multiplier", 1.)
32         self.C0 = config["initial"]["moisture_dry_basis"]
33         self.T0 = config["initial"]["temperature_C"]
34         dp = self.p["D_m2_s"]
35         self.A = dp["prefactor"] * self.options.get("D_multiplier", 1.)
36         self.a, self.B = dp["moisture_exponent"], dp["thermal_exponent_K"]
37         self.primitive = Diffusivity(1., self.a)
38         self._indices()
39
40     def _indices(self):
41         n = self.mesh.n
42         i = np.arange(n-1)
43         columns = (2*i, 2*i+1, 2*i+2, 2*i+3)

```

```

44     self.rows = np.concatenate([np.full_like(i, 0)+row for row in
45                                 (2*i, 2*i+2, 2*i+1, 2*i+3) for _ in columns]
46                                 + [np.array([2*n-2, 2*n-2, 2*n-1, 2*n-1]), 2*np.arange(n)])
47     self.cols = np.concatenate([col for _ in range(4) for col in columns]
48                                 + [np.array([2*n-2, 2*n-1, 2*n-2, 2*n-1]), 2*np.arange(n)+1])
49
50     def ambient(self, t):
51         return self.environment.value(t, "temperature"), self.beta*self.environment.value(t, "moisture")
52
53     def thermal(self, C):
54         p = self.p
55         frozen = self.options.get("freeze_thermal_moisture", False)
56         c = np.full_like(C, self.CO) if frozen else C
57         rho = p["rho_kg_m3"]["offset"] + p["rho_kg_m3"]["slope_C"]*c
58         cp = p["cp_J_kg_K"]["offset"] + p["cp_J_kg_K"]["fraction_C_over_1_plus_C"]*c/(1+c)
59         k = p["k_W_m_K"]["offset"] + p["k_W_m_K"]["fraction_C_over_1_plus_C"]*c/(1+c)
60         bp = (p["rho_kg_m3"]["slope_C"]*cp
61               + rho*p["cp_J_kg_K"]["fraction_C_over_1_plus_C"]/(1+c)**2)
62         kp = p["k_W_m_K"]["fraction_C_over_1_plus_C"]/(1+c)**2
63         if frozen:
64             bp, kp = np.zeros_like(c), np.zeros_like(c)
65         return rho*cp, k, bp, kp
66
67     def diffusion(self, C, T):
68         frozen = self.options.get("freeze_D_temperature", False)
69         temp = np.full_like(T, self.T0) if frozen else T
70         D = self.A*np.exp(-self.a/C-self.B/(temp+273.15))
71         DC = D*self.a/C**2
72         DT = np.zeros_like(D) if frozen else D*self.B/(temp+273.15)**2
73         return D, DC, DT
74
75     def surface(self, t, Ti, Ci, derivatives=False):
76         """A bracketed scalar solve after eliminating Ts(Cs); no state clipping."""
77         Ti, Ci, Ta, Ce = np.broadcast_arrays(Ti, Ci, *self.ambient(t))
78         if np.any(Ci <= 0) or np.any(Ce <= 0) or np.any(Ti+273.15 <= 0):
79             raise ValueError("Nonphysical surface input")
80         hd, md = self.h*self.mesh.half_width, self.hm*self.mesh.half_width
81         frozen = self.options.get("freeze_D_temperature", False)
82
83         def evaluate(cs):
84             _, k, _, kp = self.thermal((Ci+cs)/2)
85             ts = (k*Ti+hd*Ta)/(k+hd)
86             tm = np.full_like(ts, self.T0) if frozen else (Ti+ts)/2
87             z = md/(self.A*np.exp(-self.B/(tm+273.15)))
88             L = np.zeros_like(tm) if frozen else self.B/(tm+273.15)**2
89             ts_c = .5*kp*(Ti-ts)/(k+hd)
90             ts_t = k/(k+hd)
91             g = self.primitive.primitive_scaled(Ci)-self.primitive.primitive_scaled(cs)-z*(cs-Ce)
92             gs = -np.exp(-self.a/cs)-z + .5*z*L*ts_c*(cs-Ce)
93             return ts, g, gs, z, L, ts_c, ts_t
94
95         D = self.diffusion(Ci, Ti)[0]
96         cs = (D*Ci+md*Ce)/(D+md)
97         lo, hi = np.minimum(Ci, Ce).copy(), np.maximum(Ci, Ce).copy()
98         for _ in range(16):
99             ts, g, gs, *_ = evaluate(cs)
100             lo = np.where(g > 0, cs, lo)
101             hi = np.where(g < 0, cs, hi)
102             proposal = cs-g/gs
103             proposal = np.where((proposal < lo) | (proposal > hi), (lo+hi)/2, proposal)
104             delta = np.max(np.abs(proposal-cs))
105             cs = proposal
106             if delta < 2e-13:
107                 break

```

```

108     ts, g, gs, z, L, ts_c, ts_t = evaluate(cs)
109     bad = np.abs(g) > 3e-12
110     if np.any(bad):
111         # Rare fallback uses an independently bracketed scalar root.
112         flat = np.asarray(cs).reshape(-1).copy()
113         for j in np.flatnonzero(np.asarray(bad).reshape(-1)):
114             ti, ci, ta, ce = (float(v.flat[j]) for v in (Ti, Ci, Ta, Ce))
115             def residual(c):
116                 k = float(self.thermal(np.asarray((ci+c)/2))[1])
117                 ts_ = (k*ti+hd*ta)/(k+hd)
118                 tm_ = self.T0 if frozen else (ti+ts_)/2
119                 z_ = md/(self.A*np.exp(-self.B/(tm_+273.15)))
120                 return float(self.primitive.integral(c, ci)-z_*(c-ce))
121             flat[j] = brentq(residual, min(ci, ce), max(ci, ce), xtol=5e-14)
122         cs = flat.reshape(Ci.shape)
123         ts, g, gs, z, L, ts_c, ts_t = evaluate(cs)
124     if np.max(np.abs(g)) > 1e-10:
125         raise RuntimeError("Surface nonlinear solve failed")
126     if not derivatives:
127         return ts, cs
128     gi_t = .5*z*L*(1+ts_t)*(cs-Ce)
129     gi_c = np.exp(-self.a/Ci)+.5*z*L*ts_c*(cs-Ce)
130     cs_t, cs_c = -gi_t/gs, -gi_c/gs
131     return ts, cs, ts_t+ts_c*cs_t, ts_c*(1+cs_c), cs_t, cs_c
132
133 @staticmethod
134 def shaped(vector, ndim):
135     return vector.reshape((-1,)+(1,)*(ndim-1))
136
137 def fluxes(self, t, state, derivatives=False):
138     T, C = state[0::2], state[1::2]
139     if np.any(C <= 0) or np.any(T+273.15 <= 0):
140         raise ValueError("Nonphysical trial/accepted state; no clipping")
141     b, k, bp, kp = self.thermal(C)
142     D, DC, DT = self.diffusion(C, T)
143     m = self.mesh
144     dl, dr = (self.shaped(x, T.ndim) for x in (m.left_distances, m.right_distances))
145     rk, rd = dl/k[:-1]+dr/k[1:], dl/D[:-1]+dr/D[1:]
146     qt, qc = (T[:-1]-T[1:])/rk, (C[:-1]-C[1:])/rd
147     surf = self.surface(t, T[-1], C[-1], derivatives)
148     ta, ce = self.ambient(t)
149     qts, qcs = self.h*(surf[0]-ta), self.hm*(surf[1]-ce)
150     if not derivatives:
151         return b, bp, qt, qc, qts, qcs
152     dqt = (1/rk, qt/rk*dl*kp[:-1]/k[:-1]**2,
153            -1/rk, qt/rk*dr*kp[1:]/k[1:]**2)
154     dqc = (qc/rd*dl*DT[:-1]/D[:-1]**2,
155            1/rd+qc/rd*dl*DC[:-1]/D[:-1]**2,
156            qc/rd*dr*DT[1:]/D[1:]**2,
157            -1/rd+qc/rd*dr*DC[1:]/D[1:]**2)
158     return b, bp, qt, qc, qts, qcs, dqt, dqc, surf
159
160 def rhs(self, t, state):
161     b, _, qt, qc, qts, qcs = self.fluxes(t, state)
162     area = self.shaped(self.mesh.faces[1:-1], qt.ndim)
163     volume = self.shaped(self.mesh.volumes, qt.ndim)
164     out = np.empty_like(state)
165     for index, flux, surface, capacity in ((0, qt, qts, b), (1, qc, qcs, 1.)):
166         total = np.concatenate((np.zeros_like(flux[1:]), area*flux,
167                                np.asarray(self.mesh.radius*surface).reshape((1,)+flux.shape[1:])), axis=0)
168         out[index::2] = -np.diff(total, axis=0)/(volume*capacity)
169     return out
170
171 def jacobian(self, t, state):

```

```

172     b, bp, qt, qc, qts, qcs, dqt, dqc, surf = self.fluxes(t, state, True)
173     m = self.mesh
174     a = m.faces[1:-1]
175     values = ([ -a*d/(m.volumes[:-1]*b[:-1]) for d in dqt]
176               + [a*d/(m.volumes[1:]*b[1:]) for d in dqt]
177               + [-a*d/m.volumes[:-1] for d in dqc]
178               + [a*d/m.volumes[1:] for d in dqc])
179     values.append(-m.radius/m.volumes[-1]*np.array([
180         self.h*surf[2]/b[-1], self.h*surf[3]/b[-1], self.hm*surf[4], self.hm*surf[5]]))
181     total = np.r_[0., a*qt, m.radius*qts]
182     tdot = -np.diff(total)/(m.volumes*b)
183     values.append(-tdot*bp/b)
184     return coo_matrix((np.concatenate(values), (self.rows, self.cols)),
185                      shape=(2*m.n, 2*m.n)).tocsc()

```

文件: src/coupled/integrate.py

Listing 14:

```

1  """Segmented coupled integration, streaming reconstruction and balance checks."""
2  import time
3
4  import numpy as np
5  from numpy.polynomial.legendre import leggauss
6  from scipy.integrate import solve_ivp
7  from scipy.interpolate import PchipInterpolator
8
9  from src.solver import InitializedBDF
10
11
12  def reconstruct(model, times, state, radii):
13      """PCHIP has no overshoot inside each interval; axis uses an even quadratic."""
14      mesh = model.mesh
15      T, C = state[0::2].T, state[1::2].T
16      surfaces = model.surface(times, T[:, -1], C[:, -1])
17      x0, x1 = mesh.centres[:2]
18      points = np.r_[0., mesh.centres, mesh.radius]
19      fields, extended = [], []
20      for cells, surface, initial in zip((T, C), surfaces, (model.T0, model.C0)):
21          axis = (x1*x1*cells[:, 0]-x0*x0*cells[:, 1])/(x1*x1-x0*x0)
22          nodes = np.column_stack((axis, cells, surface))
23          sampled = PchipInterpolator(points, nodes, axis=1)(radii)
24          near = radii < x0
25          sampled[:, near] = axis[:, None]+((cells[:, 1]-cells[:, 0])/(x1*x1-x0*x0))[:, None]*radii[near]**2
26          sampled[np.asarray(times) == 0] = initial
27          nodes[np.asarray(times) == 0] = initial
28          fields.append(sampled)
29          extended.append(nodes)
30      # The central even quadratic and intervalwise PCHIP are bounded by endpoints.
31      maximum_index = np.argmax(extended[1], axis=1)
32      diagnostics = {
33          "temperature_mean": np.sum(T*mesh.weights, axis=1),
34          "moisture_mean": np.sum(C*mesh.weights, axis=1),
35          "moisture_global_max": np.max(extended[1], axis=1),
36          "moisture_argmax_radius_m": points[maximum_index],
37          "moisture_radial_increase_max": np.maximum(0., np.max(np.diff(extended[1], axis=1), axis=1)),
38      }
39      return fields, diagnostics
40
41
42  def integrate(model, end=10800., rtol=1e-9, atol_temperature=1e-10,
43              atol_moisture=1e-12, max_step=10., method="BDF", output_step=1.):
44      started = time.perf_counter()

```

```

45 n = model.mesh.n
46 initial = np.empty(2*n)
47 initial[0::2], initial[1::2] = model.T0, model.CO
48 state = initial.copy()
49 times = np.unique(np.r_[np.arange(0., end+1e-9, output_step), end])
50 radii = np.arange(21)*.001
51 cache = {"time_s": times, "radius_m": radii,
52          "cell_centres_m": model.mesh.centres, "faces_m": model.mesh.faces,
53          "weights": model.mesh.weights, "temperature": np.empty((len(times), 21)),
54          "moisture": np.empty((len(times), 21))}
55 knots = model.environment.times
56 cuts = np.unique(np.r_[0., knots[(knots > 0) & (knots < end)], end])
57 profile_times, profiles = [0.], [initial.copy()]
58 gx, gw = leggauss(8)
59 water_out = heat_out = 0.
60 water_error = heat_error = semi_heat_error = 0.
61 accepted, nfev, njev, nlu = [], 0, 0, 0
62 minimum_C, minimum_T = model.CO, model.T0
63 b0 = float(model.thermal(np.asarray(model.CO))[0])
64 atol = np.tile([atol_temperature, atol_moisture], n)
65 for a, b in zip(cuts[:-1], cuts[1:]):
66     integrator = InitializedBDF if method == "BDF" else method
67     result = solve_ivp(model.rhs, (a, b), state, method=integrator, rtol=rtol,
68                        atol=atol, jac=model.jacobian, dense_output=True, max_step=max_step)
69     if not result.success or not np.isfinite(result.y).all():
70         raise RuntimeError(result.message)
71     if np.min(result.y[1::2]) <= 0:
72         raise RuntimeError("Nonpositive accepted moisture")
73     state = result.y[:, -1].copy()
74     minimum_C = min(minimum_C, float(np.min(result.y[1::2])))
75     minimum_T = min(minimum_T, float(np.min(result.y[0::2])))
76     indices = np.flatnonzero((times >= a) & (times <= b))
77     if len(indices):
78         fields, extra = reconstruct(model, times[indices], result.sol(times[indices]), radii)
79         for name, values in zip(("temperature", "moisture"), fields):
80             cache[name][indices] = values
81         for key, values in extra.items():
82             if key not in cache:
83                 cache[key] = np.empty(len(times))
84             cache[key][indices] = values
85     steps = np.diff(result.t)
86     accepted.extend(steps.tolist())
87     nfev += result.nfev
88     njev += result.njev
89     nlu += result.nlu
90     # Independent quadrature of the reconstructed boundary-flow history.
91     # Variable-b heat balance uses  $H = \text{mean}[b(C)*(T-T_0)]$ , minus its  $b'(C)$ 
92     # correction, not an erroneous uncorrected difference of  $\rho \cdot c_p \cdot T$ .
93     for first in range(0, len(steps), 64):
94         last = min(first+64, len(steps))
95         dt = steps[first:last]
96         mid = .5*(result.t[first+1:last+1]+result.t[first:last])
97         tq = (mid[:, None]+.5*dt[:, None]*gx).ravel()
98         yq = result.sol(tq)
99         dyq = model.rhs(tq, yq)
100         capacity, _, bp, _ = model.thermal(yq[1::2])
101         ts, cs = model.surface(tq, yq[-2], yq[-1])
102         ta, ce = model.ambient(tq)
103         qt, qc = model.h*(ts-ta), model.hm*(cs-ce)
104         correction = np.sum(model.mesh.weights[:, None]*bp*(yq[0::2]-model.T0)*dyq[1::2], axis=0)
105         heat_rate = 2*qt/model.mesh.radius-correction
106         water_rate = 2*qc/model.mesh.radius
107         heat_integral = heat_out+np.cumsum(.5*dt*np.sum(heat_rate.reshape(-1, 8)*gw, axis=1))
108         water_integral = water_out+np.cumsum(.5*dt*np.sum(water_rate.reshape(-1, 8)*gw, axis=1))

```

```

109     yend = result.y[:, first+1:last+1]
110     bend = model.thermal(yend[1::2])[0]
111     heat_content = np.sum(model.mesh.weights[:, None]*bend*(yend[0::2]-model.T0), axis=0)
112     water_mean = np.sum(model.mesh.weights[:, None]*yend[1::2], axis=0)
113     heat_error = max(heat_error, float(np.max(np.abs(heat_content+heat_integral))))
114     water_error = max(water_error, float(np.max(np.abs(water_mean-model.CO+water_integral))))
115     heat_out, water_out = float(heat_integral[-1]), float(water_integral[-1])
116     semi = np.sum(model.mesh.weights[:, None]*capacity*dyq[0::2], axis=0)+2*qt/model.mesh.radius
117     semi_heat_error = max(semi_heat_error, float(np.max(np.abs(semi))))
118     profile_times.append(float(b))
119     profiles.append(state.copy())
120     cache["profile_time_s"] = np.asarray(profile_times)
121     cache["profile_state"] = np.asarray(profiles)
122     cache["final_state"] = state
123     stats = {
124         "accepted_steps": len(accepted), "accepted_max_step_s": max(accepted),
125         "accepted_min_step_s": min(accepted), "nfev": nfev, "njev": njev, "nlu": nlu,
126         "minimum_accepted_temperature_C": minimum_T, "minimum_accepted_moisture": minimum_C,
127         "balance_quadrature_order": 8,
128         "water_balance_max_abs_dry_basis": water_error,
129         "water_balance_normalizer": model.CO, "water_balance_relative": water_error/model.CO,
130         "heat_corrected_balance_max_abs_J_m3": heat_error,
131         "heat_balance_normalizer_J_m3": b0*22., "heat_corrected_balance_relative": heat_error/(b0*22.),
132         "heat_semidiscrete_balance_max_abs_W_m3": semi_heat_error,
133         "heat_balance_definition": "mean[b(C)*(T-T0)] + integral(2*qT/R - mean[bprime*(T-T0)*Cdot])",
134         "elapsed_s": time.perf_counter()-started,
135     }
136     return cache, stats

```

文件: src/events/__init__.py

Listing 15:

```

1  """Whole-domain drying events, independently versioned from Q1/Q2 kernels."""

```

文件: src/events/integration.py

Listing 16:

```

1  """Continue an unrounded Q23 state and locate its reconstructed global event.
2
3  No quasi-isothermal replacement is made. Output is reconstructed in chunks;
4  only occasional internal profiles are retained. The 4 h discontinuity is an
5  explicit left/right split, not a rapidly interpolated physical transient.
6  """
7  import time
8
9  import numpy as np
10 from numpy.polynomial.legendre import leggauss
11 from scipy.integrate import solve_ivp
12
13 from src.boundary import Environment
14 from src.coupled.integrate import reconstruct
15 from src.solver import InitializedBDF
16
17
18 class ScenarioEnvironment(Environment):
19     def __init__(self, config, scenario="main"):
20         super().__init__(config)
21         if scenario == "last_observation":
22             self.platform = self.data[-1, 1:].copy()
23         elif scenario == "mean_9000_14400":

```

```

24         self.platform = self.data[self.times >= 9000, 1:].mean(axis=0)
25     elif scenario != "main":
26         raise ValueError("Unknown environment extension")
27     self.scenario = scenario
28     self.right_at_join = False
29
30     def value(self, t, field):
31         t = np.asarray(t)
32         value = super().value(t, field)
33         if self.right_at_join:
34             col = 0 if field == "temperature" else 1
35             platform = self.platform[col] * (1 if col == 0 else self.beta)
36             value = np.where(t >= self.times[-1], platform, value)
37         return value
38
39
40     def global_maximum(model, t, state):
41         """Includes even-axis value, every cell and the actual surface.
42
43         PCHIP cannot overshoot interval endpoints. The even quadratic on the first
44         half-cell is monotone in  $r^2$ , so its maximum is also at an endpoint.
45         """
46         C = state[1::2]
47         x0, x1 = model.mesh.centres[:2]
48         axis = (x1*x1*C[0]-x0*x0*C[1])/(x1*x1-x0*x0)
49         surface = model.surface(t, state[-2], state[-1])[1]
50         values = np.concatenate((np.asarray(axis).reshape((1,)+C.shape[1:]), C,
51                                np.asarray(surface).reshape((1,)+C.shape[1:])), axis=0)
52         return np.max(values, axis=0)
53
54
55     def continue_to_event(model, initial_state, start=10800., limit=604800.,
56                          rtol=1e-10, atol_temperature=1e-11, atol_moisture=1e-13,
57                          max_step=60., method="BDF", output_step=60., chunk_s=1800.):
58         if start != 10800. or limit <= start:
59             raise ValueError("Continuation must use the 3 h Q2 checkpoint and original clock")
60         begun = time.perf_counter()
61         n, R = model.mesh.n, model.mesh.radius
62         initial_state = np.asarray(initial_state).copy()
63         if initial_state.shape != (2*n,) or not np.isfinite(initial_state).all():
64             raise ValueError("Checkpoint dimension/state mismatch")
65         threshold = model.config["events"]["moisture_threshold"]
66         if global_maximum(model, start, initial_state) <= threshold:
67             raise ValueError("Checkpoint already meets threshold; earlier event must be searched")
68         state = initial_state.copy()
69         atol = np.tile([atol_temperature, atol_moisture], n)
70         integrator = InitializedBDF if method == "BDF" else method
71         radii = np.arange(21)*.001
72         times_out, Ts, Cs, extras = [], [], [], {}
73         profile_t, profile_y = [start], [state.copy()]
74         cuts = np.unique(np.r_[start, model.environment.times[(model.environment.times > start)],
75                               np.arange(14400., limit, chunk_s), limit])
76         cuts = cuts[(cuts >= start) & (cuts <= limit)]
77         gx, gw = leggauss(8)
78         heat_out = water_out = 0.
79         water_error = heat_error = 0.
80         binit = model.thermal(state[1::2])[0]
81         Hinit = float(np.sum(model.mesh.weights*binit*(state[::2]-model.T0)))
82         Cinit = float(np.sum(model.mesh.weights*state[1::2]))
83         initial_capacity = float(model.thermal(np.asarray(model.CO))[0])
84         accepted, accepted_maximum, accepted_times = [], [], []
85         nfev = nlu = njev = 0
86         max_temporal_increase = 0.
87         event_time = event_state = bracket = None

```

```

88     minimum_C = float(np.min(state[1::2]))
89
90     def event(t, y):
91         return float(global_maximum(model, t, y)-threshold)
92     event.direction, event.terminal = -1, True
93
94     for a, b in zip(cuts[:-1], cuts[1:]):
95         model.environment.right_at_join = a >= 14400.
96         result = solve_ivp(model.rhs, (a, b), state, method=integrator, rtol=rtol,
97                             atol=atol, jac=model.jacobian, dense_output=True,
98                             max_step=max_step, events=event)
99         if not result.success or not np.isfinite(result.y).all():
100             raise RuntimeError(result.message)
101         state = result.y[:, -1].copy()
102         stop = float(result.t[-1])
103         minimum_C = min(minimum_C, float(np.min(result.y[1::2])))
104         if minimum_C <= 0:
105             raise RuntimeError("Nonpositive accepted moisture")
106         steps = np.diff(result.t)
107         accepted.extend(steps.tolist())
108         nfev += result.nfev; nlu += result.nlu; njev += result.njev
109         # Accepted nodes plus midpoints: check spatial maxima, no centre-only surrogate.
110         check_times = np.sort(np.r_[result.t, .5*(result.t[:-1]+result.t[1:])])
111         check_max = global_maximum(model, check_times, result.sol(check_times))
112         max_temporal_increase = max(max_temporal_increase, float(np.max(np.diff(check_max))))
113         accepted_times.extend(check_times.tolist())
114         accepted_maximum.extend(check_max.tolist())
115         tsample = np.arange(np.floor(a/output_step)*output_step+output_step, stop+1e-9, output_step)
116         tsample = tsample[(tsample > a) & (tsample <= stop)]
117         if a == start:
118             tsample = np.r_[start, tsample]
119         if result.t_events[0].size:
120             event_time = float(result.t_events[0][0])
121             event_state = result.y_events[0][0].copy()
122             if not len(tsample) or abs(tsample[-1]-event_time) > 1e-9:
123                 tsample = np.r_[tsample, event_time]
124             else:
125                 tsample[-1] = event_time
126             bracket = [float(result.t[-2]), event_time]
127         # Bounded-memory spatial reconstruction of each output chunk.
128         if len(tsample):
129             fields, diag = reconstruct(model, tsample, result.sol(tsample), radii)
130             times_out.append(tsample); Ts.append(fields[0]); Cs.append(fields[1])
131             for key, value in diag.items():
132                 extras.setdefault(key, []).append(value)
133         for first in range(0, len(steps), 32):
134             last = min(first+32, len(steps))
135             dt = steps[first:last]
136             mid = .5*(result.t[first+1:last+1]+result.t[first:last])
137             tq = (mid[:, None]+.5*dt[:, None]*gx).ravel()
138             yq = result.sol(tq)
139             dyq = model.rhs(tq, yq)
140             bp = model.thermal(yq[1::2])[2]
141             st, sc = model.surface(tq, yq[-2], yq[-1])
142             ta, ce = model.ambient(tq)
143             correction = np.sum(model.mesh.weights[:, None]*bp*(yq[1::2]-model.T0)*dyq[1::2], axis=0)
144             hrate = 2*model.h/R*(st-ta)-correction
145             crate = 2*model.hm/R*(sc-ce)
146             hi = heat_out+np.cumsum(.5*dt*np.sum(hrate.reshape(-1, 8)*gw, axis=1))
147             ci = water_out+np.cumsum(.5*dt*np.sum(crate.reshape(-1, 8)*gw, axis=1))
148             ye = result.y[:, first+1:last+1]
149             H = np.sum(model.mesh.weights[:, None]*model.thermal(ye[1::2])[0]*(ye[1::2]-model.T0), axis=0)
150             mean = np.sum(model.mesh.weights[:, None]*ye[1::2], axis=0)
151             heat_error = max(heat_error, float(np.max(np.abs(H-Hinit+hi))))

```

```

152         water_error = max(water_error, float(np.max(np.abs(mean-Cinit+ci))))
153         heat_out, water_out = float(hi[-1]), float(ci[-1])
154     profile_t.append(stop); profile_y.append(state.copy())
155     if result.t_events[0].size:
156         break
157     if b % 21600 == 0:
158         print(f"t={b/3600:.1f} h, full-domain Cmax={check_max[-1]:.7f}", flush=True)
159
160     cache = {"time_s": np.concatenate(times_out), "radius_m": radii,
161             "temperature": np.concatenate(Ts), "moisture": np.concatenate(Cs),
162             "profile_time_s": np.asarray(profile_t), "profile_state": np.asarray(profile_y),
163             "final_state": state, "cell_centres_m": model.mesh.centres,
164             "faces_m": model.mesh.faces, "weights": model.mesh.weights,
165             "accepted_check_time_s": np.asarray(accepted_times),
166             "accepted_check_global_max": np.asarray(accepted_maximum)}
167     cache.update({key: np.concatenate(value) for key, value in extras.items()})
168     stats = {"accepted_steps": len(accepted), "accepted_max_step_s": max(accepted),
169             "accepted_min_step_s": min(accepted), "nfev": nfev, "nlu": nlu, "njev": njev,
170             "minimum_accepted_moisture": minimum_C,
171             "global_max_temporal_increase_at_checked_points": max_temporal_increase,
172             "water_balance_tail_max_abs": water_error, "water_balance_relative": water_error/model.CO,
173             "heat_corrected_balance_tail_max_abs_J_m3": heat_error,
174             "heat_corrected_balance_relative": heat_error/(initial_capacity*22),
175             "event_found": event_time is not None, "event_time_s": event_time,
176             "boundary_extension": model.environment.scenario,
177             "boundary_platform_T_Ceq": [float(model.environment.platform[0]), float(model.environment.platform[1]*model.
178             environment.beta*model.beta)],
179             "event_search": "max(all_cells,even_axis,true_surface), PCHIP bounded by nodes",
180             "event_bracket_s": bracket, "balance_quadrature_order": 8}
181     if event_time is not None:
182         # Independently continue beyond the zero instead of calling it strictly dry.
183         later = float(np.floor(event_time)+1)
184         after = max(event_time+1., later)
185         post = solve_ivp(model.rhs, (event_time, after), event_state, method=integrator,
186             rtol=rtol, atol=atol, jac=model.jacobian, dense_output=True,
187             max_step=min(1., max_step))
188         if not post.success:
189             raise RuntimeError("Post-event verification failed")
190         pre_time = max(result.t[0], event_time-1.)
191         check_t = np.asarray([pre_time, event_time, later, after])
192         values = [event(pre_time, result.sol(pre_time)), event(event_time, event_state),
193             event(later, post.sol(later)), event(after, post.y[:, -1])]
194         assert values[0] > 0 and values[2] < 0 and values[3] < 0
195         slope = (values[-1]-values[0])/(after-pre_time)
196         stats.update({"post_event_times_s": check_t.tolist(), "post_event_g": values,
197             "event_slope_per_s": float(slope), "first_checked_integer_second": later,
198             "strict_inequality_holds_at_critical_time": False})
199         cache["post_event_state"] = post.y[:, -1]
200         cache["event_state"] = event_state
201         stats["elapsed_s"] = time.perf_counter()-begun
202     return cache, stats

```

文件: src/moving/__init__.py

Listing 17:

```

1  """Affine material-coordinate Q4, separately versioned from Q1/Q23."""

```

文件: src/moving/model.py

Listing 18:

```

1  """Moving-domain transport on a fixed reference-radius mesh.
2
3  Let  $s=R(t)/R_0$ . The reference operator is multiplied by  $s^{-2}$ , and
4  reference Robin coefficients are  $s \cdot h$  and  $s \cdot h_m$ . No  $R_{dot}$  advection is added:
5  material velocity equals grid velocity. Dry-basis state is not diluted.
6  """
7  from pathlib import Path
8  import numpy as np
9  from openpyxl import load_workbook
10 from scipy.interpolate import PchipInterpolator
11
12 from src.coupled.model import CoupledFVM
13
14
15 class Radius:
16     def __init__(self, config, kind="linear", extension="error"):
17         if kind not in ("linear", "pchip", "fixed") or extension not in ("error", "hold_last"):
18             raise ValueError("Unknown radius scenario")
19         self.kind, self.extension = kind, extension
20         self.R0 = config["geometry"]["reference_radius_m"]
21         wb = load_workbook(Path(config["source_directory"])/"附件2.xlsx", read_only=True, data_only=True)
22         self.data = np.asarray(list(wb.active.values)[1:], float); wb.close()
23         self.times = self.data[:, 0]
24         self.radii = self.data[:, 1] * .01
25         self.curve = PchipInterpolator(self.times, self.radii, extrapolate=False)
26         assert self.times[0] == 0 and self.radii[0] == self.R0
27
28     def value(self, t):
29         t = np.asarray(t, dtype=float)
30         if np.any(t < 0):
31             raise ValueError("Negative radius time")
32         if self.kind == "fixed":
33             return np.full_like(t, self.R0)
34         if np.any(t > self.times[-1]) and self.extension == "error":
35             raise ValueError("Radius observations end at 72 h; no unlabelled extrapolation")
36         tq = np.minimum(t, self.times[-1])
37         return np.interp(tq, self.times, self.radii) if self.kind == "linear" else self.curve(tq)
38
39
40 class MovingCoupled(CoupledFVM):
41     def __init__(self, mesh, config, environment, radius, group="q4", perturbation=None):
42         super().__init__(mesh, config, environment, group, perturbation)
43         self.physical_h, self.physical_hm = self.h, self.hm
44         self.radius = radius
45
46     def geometry(self, t):
47         ratio = self.radius.value(t) / self.mesh.radius
48         self.h = self.physical_h * ratio
49         self.hm = self.physical_hm * ratio
50         return ratio
51
52     def surface(self, t, Ti, Ci, derivatives=False):
53         self.geometry(t)
54         if self.A != 0:
55             return super().surface(t, Ti, Ci, derivatives)
56         if self.physical_hm != 0:
57             raise ValueError("Zero diffusion test requires insulated water boundary")
58         Ti, Ci, Ta, _ = np.broadcast_arrays(Ti, Ci, *self.ambient(t))
59         _, k, _, kp = self.thermal(Ci)
60         hd = self.h * self.mesh.half_width
61         Ts = (k * Ti + hd * Ta) / (k + hd)
62         if not derivatives:
63             return Ts, Ci.copy()
64         return Ts, Ci.copy(), k / (k + hd), kp * (Ti - Ts) / (k + hd), np.zeros_like(Ci), np.ones_like(Ci)

```

```

65
66 def fluxes(self,t,state,derivatives=False):
67     self.geometry(t)
68     if self.A!=0:
69         return super().fluxes(t,state,derivatives)
70     T,C=state[:,2],state[1:,2]
71     if np.any(C<=0) or np.any(T+273.15<=0):
72         raise ValueError("Nonphysical test state")
73     b,k,bp,kp=self.thermal(C)
74     dl,dr=(self.shaped(x,T.ndim) for x in (self.mesh.left_distances,self.mesh.right_distances))
75     resistance=dl/k[:-1]+dr/k[1:]
76     qt=(T[:-1]-T[1:])/resistance
77     surf=self.surface(t,T[:-1],C[:-1],derivatives)
78     qts=self.h*(surf[0]-self.ambient(t)[0])
79     qc=np.zeros_like(qt); qcs=np.zeros_like(qts)
80     if not derivatives:
81         return b,bp,qt,qc,qts,qcs
82     dqt=(1/resistance,qt/resistance*dl*kp[:-1]/k[:-1]**2,
83          -1/resistance,qt/resistance*dr*kp[1:]/k[1:]**2)
84     return b,bp,qt,qc,qts,qcs,dqt,(qc,qc,qc,qc),surf
85
86 def rhs(self,t,state):
87     ratio=self.geometry(t)
88     return super().rhs(t,state)/ratio**2
89
90 def jacobian(self,t,state):
91     ratio=self.geometry(t)
92     return super().jacobian(t,state)/ratio**2

```

文件: src/moving/integration.py

Listing 19:

```

1  """Fresh-start moving Q4 with true physical-coordinate output and balances."""
2  import time
3  import numpy as np
4  from numpy.polynomial.legendre import leggauss
5  from scipy.integrate import solve_ivp
6  from scipy.interpolate import PchipInterpolator
7
8  from src.events.integration import global_maximum
9  from src.solver import InitializedBDF
10
11
12 def reconstruct(model,times,state):
13     times=np.asarray(times)
14     T,C=state[:,2].T,state[1:,2].T
15     R=model.radius.value(times)
16     surfaces=model.surface(times,T[:,-1],C[:,-1])
17     R0=model.mesh.radius
18     centres=model.mesh.centres/R0
19     x0,x1=centres[:,2]
20     points=np.r_[0.,centres,1.]
21     radii=np.arange(21)*.001
22     values=[]; extras={"radius_current_m":R}
23     for name,cells,surface,initial in zip(("temperature","moisture"),(T,C),surfaces,(model.T0,model.C0)):
24         axis=(x1*x1*cells[:,0]-x0*x0*cells[:,1])/(x1*x1-x0*x0)
25         nodes=np.column_stack((axis,cells,surface))
26         nodes[times==0]=initial
27         array=np.full((len(times),22),np.nan)
28         for i in range(len(times)):
29             valid=radii<=R[i]
30             xi=radii[valid]/R[i]

```

```

31         curve=PchipInterpolator(points,nodes[i],extrapolate=False)
32         sampled=curve(xi)
33         near=xi<x0
34         sampled[near]=nodes[i,0]+(cells[i,1]-cells[i,0])/(x1*x1-x0*x0)*xi[near]**2
35         array[i,np.flatnonzero(valid)]=sampled
36         array[i,-1]=nodes[i,-1]
37         values.append(array)
38         extras[name+"_mean"]=np.sum(cells*model.mesh.weights,axis=1)
39         if name=="moisture":
40             extras["moisture_global_max"]=np.max(nodes,axis=1)
41             extras["moisture_argmax_xi"]=points[np.argmax(nodes,axis=1)]
42             extras["moisture_radial_increase_max"]=np.maximum(0,np.max(np.diff(nodes,axis=1),axis=1))
43         return values,extras
44
45
46 def integrate(model,limit=259200.,rtol=1e-10,atol_temperature=1e-11,
47             atol_moisture=1e-13,max_step=30.,method="BDF",output_step=60.):
48     started=time.perf_counter()
49     n=model.mesh.n
50     initial=np.tile([model.T0,model.CO],n)
51     state=initial.copy()
52     # Explicitly demand supported geometry for the entire requested horizon.
53     model.radius.value(limit)
54     cuts=np.unique(np.r_[0.,model.environment.times,model.radius.times,
55                         np.arange(0.,limit,1800.),limit])
56     cuts=cuts[(cuts>=0)&(cuts<=limit)]
57     times,fields,extras=[],[],[],{}
58     profile_t,profile_y=[0.],[initial.copy()]
59     accepted_t,accepted_max,steps_all=[],[],[]
60     gx,gw=leggauss(8)
61     water_out=heat_out=water_error=heat_error=0.
62     cap0=float(model.thermal(np.asarray(model.CO))[0])
63     counts={"nfev":0,"njev":0,"nlu":0}
64     event_time=None; event_state=None
65     minC=model.CO
66     atol=np.tile([atol_temperature,atol_moisture],n)
67     solver=InitializedBDF if method=="BDF" else method
68
69     def event(t,y):
70         return float(global_maximum(model,t,y)-model.config["events"]["moisture_threshold"])
71     event.direction,event.terminal=-1,True
72
73     for a,b in zip(cuts[:-1],cuts[1:]):
74         model.environment.right_at_join=a>=14400.
75         result=solve_ivp(model.rhs,(a,b),state,method=solver,rtol=rtol,atol=atol,
76                         jac=model.jacobian,dense_output=True,max_step=max_step,events=event)
77         if not result.success or not np.isfinite(result.y).all():
78             raise RuntimeError(result.message)
79         state=result.y[:,-1].copy()
80         stop=float(result.t[-1])
81         minC=min(minC,float(np.min(result.y[1::2])))
82         if minC<=0:
83             raise RuntimeError("Nonpositive accepted moisture")
84         dt=np.diff(result.t); steps_all.extend(dt.tolist())
85         for k in counts: counts[k]+=getattr(result,k)
86         checks=np.sort(np.r_[result.t,.5*(result.t[:-1]+result.t[1:])])
87         maxima=global_maximum(model,checks,result.sol(checks))
88         accepted_t.extend(checks.tolist()); accepted_max.extend(maxima.tolist())
89         ts=np.arange(np.floor(a/output_step)*output_step+output_step,stop+1e-9,output_step)
90         ts=ts[(ts>a)&(ts<=stop)]
91         if a==0:
92             ts=np.unique(np.r_[0.,[v for v in (1.,10.) if v<=stop],ts])
93         if result.t_events[0].size:
94             event_time=float(result.t_events[0][0]); event_state=result.y_events[0][0].copy()

```

```

95         if len(ts) and abs(ts[-1]-event_time)<1e-9: ts[-1]=event_time
96         else: ts=np.r_[ts,event_time]
97     elif b==limit and (not len(ts) or ts[-1]!=b):
98         ts=np.r_[ts,b]
99     if len(ts):
100         sampled,diag=reconstruct(model,ts,result.sol(ts))
101         times.append(ts)
102         for j in range(2): fields[j].append(sampled[j])
103         for k,v in diag.items(): extras.setdefault(k,[]).append(v)
104     for first in range(0,len(dt),32):
105         last=min(first+32,len(dt)); widths=dt[first:last]
106         mid=.5*(result.t[first+1:last+1]+result.t[first:last])
107         tq=(mid[:,None]+.5*widths[:,None]*gx).ravel()
108         yq=result.sol(tq); dy=model.rhs(tq,yq)
109         st,sc=model.surface(tq,yq[-2],yq[-1]); ta,ce=model.ambient(tq)
110         R=model.radius.value(tq)
111         correction=np.sum(model.mesh.weights[:,None]*model.thermal(yq[1::2])[2]*(yq[1::2]-model.T0)*dy[1::2],axis=0)
112         heat_rate=2*model.physical_h/R*(st-ta)-correction
113         water_rate=2*model.physical_hm/R*(sc-ce)
114         hi=heat_out+np.cumsum(.5*widths*np.sum(heat_rate.reshape(-1,8)*gw,axis=1))
115         wi=water_out+np.cumsum(.5*widths*np.sum(water_rate.reshape(-1,8)*gw,axis=1))
116         ye=result.y[:,first+1:last+1]
117         H=np.sum(model.mesh.weights[:,None]*model.thermal(ye[1::2])[0]*(ye[1::2]-model.T0),axis=0)
118         Cmean=np.sum(model.mesh.weights[:,None]*ye[1::2],axis=0)
119         heat_error=max(heat_error,float(np.max(np.abs(H+hi))))
120         water_error=max(water_error,float(np.max(np.abs(Cmean-model.C0+wi))))
121         heat_out,water_out=float(hi[-1]),float(wi[-1])
122     profile_t.append(stop); profile_y.append(state.copy())
123     if event_time is not None: break
124     if b%21600==0:
125         print(f"Q4 {model.group}/{model.radius.kind}, {b/3600:.0f} h: R={model.radius.value(b)*100:.5f} cm, Cmax={
126             maxima[-1]:.7f}",flush=True)
127     cache={"time_s":np.concatenate(times),"radius_fixed_m":np.arange(21)*.001,
128         "temperature":np.concatenate(fields[0]),"moisture":np.concatenate(fields[1]),
129         "profile_time_s":np.asarray(profile_t),"profile_state":np.asarray(profile_y),
130         "final_state":state,"cell_centres_reference_m":model.mesh.centres,
131         "reference_faces_m":model.mesh.faces,"weights":model.mesh.weights,
132         "accepted_check_time_s":np.asarray(accepted_t),"accepted_check_global_max":np.asarray(accepted_max)}
133     cache.update({k:np.concatenate(v) for k,v in extras.items()})
134     stats={**counts,"accepted_steps":len(steps_all),"accepted_max_step_s":max(steps_all),
135         "accepted_min_step_s":min(steps_all),"minimum_accepted_moisture":minC,
136         "water_balance_max_abs":water_error,"water_balance_relative":water_error/model.C0,
137         "heat_corrected_balance_max_abs_J_m3":heat_error,"heat_corrected_balance_relative":heat_error/(cap0*22),
138         "heat_balance_definition":"normalized mean[b*(T-T0)] + integral(2*qT/R - mean[bprime*(T-T0)*Cdot]); no duplicate
139         geometry term",
140         "event_found":event_time is not None,"event_time_s":event_time,
141         "end_time_s":stop,"radius_end_m":float(model.radius.value(stop)),
142         "global_max_at_end":float(cache["moisture_global_max"][-1]),
143         "domain_supported_through_s":float(model.radius.times[-1]),
144         "radius_extension":model.radius.extension,"global_max_increase_at_checks":float(max(0,np.max(np.diff(accepted_max))
145     ))}
146     if event_time is not None:
147         after=event_time+1.
148         if model.radius.kind!="fixed" and model.radius.extension=="error" and after>model.radius.times[-1]:
149             raise RuntimeError("Insufficient observed radius for post-event verification")
150         post=solve_ivp(model.rhs,(event_time,after),event_state,method=solver,rtol=rtol,
151             atol=atol,jac=model.jacobian,dense_output=True,max_step=min(1.,max_step))
152         if not post.success: raise RuntimeError("Post-event solve failed")
153         before=max(result.t[0],event_time-1.)
154         integer=float(np.floor(event_time)+1)
155         values=[event(before,result.sol(before)),event(event_time,event_state),
156             event(integer,post.sol(integer)),event(after,post.y[:,-1])]
157         assert values[0]>0 and values[2]<0 and values[3]<0
158         stats.update({"post_event_time_s":[before,event_time,integer,after],"post_event_g":values,

```

```
156         "event_slope_per_s":(values[-1]-values[0])/(after-before),"first_strict_integer_second":integer})
157     cache["event_state"]=event_state; cache["post_event_state"]=post.y[:,-1]
158     stats["elapsed_s"]=time.perf_counter()-started
159     return cache,stats
```